TinyServe: the core study guide

From tokens to serving and honest benchmarks

TinyServe learning project

Contents

How to use this study guide 4
Reading order 5
PDF and reproducible build 5
01. Tokens, tensors, shapes, logits (Lesson A1) 6
The pipeline 6
Ideas to remember 6
Code map 6
Interview questions 6
02. GPU basics: bandwidth, matmul, roofline (Lesson A2) 8
Three ideas 8
What we measured on the T4 (FP16, ranges over 7 trials) 8
Batching, measured 8
Decode ceiling for TinyLlama 9
Code map 9
Interview questions 9
03. The Hugging Face `generate` baseline (Lesson A3) 11
Definitions (kept for our own engine) 11
Results 11
What the numbers say 11
Targets for our engine 11
Code map 12
Interview questions 12
04. Weight tour (Lesson B1) 13
The 201 tensors 13
Where the bytes are 13
GQA and the KV cache 13
What the weights look like (FP32 statistics) 14
Code map 14
Interview questions 14
05. RMSNorm (Lesson B2) 15
The operation 15
Implementation details that matter 15
Test results 15
What the real activations look like 15
Code map 16
Interview questions 16
06. Rotary position embeddings (B3) 17
Give attention position information 17
Hand example 17
Fast and slow pairs 17
Why the score depends on relative position 17
Shapes, dtype and cache implications 18
Recorded validation 18

Interview questions 18
07. Attention scores, causal mask and softmax (B4) 19
The four steps 19
Why divide by sqrt(head_dim)? 19
Shapes and causality 19
Precision and validation 19
Interview questions 20
08. Grouped-query attention (B5) 21
Share K/V, not queries or attention weights 21
Implementation and shapes 21
Where the memory saving comes from 21
Verification 21
Interview questions 22
09. Llama MLP and SwiGLU (B6) 23
One token at a time 23
Hand example 23
Weights and shapes 23
Verification and numerics 23
Interview questions 24
10. Decoder layer and residuals (B7) 25
Connect the pieces 25
Attention branch 25
Flattened metadata 25
Verification 25
Interview questions 26
11. Full model, loading and FP32 parity (B8) 27
Complete path 27
Weight loading 27
What B8 verifies beyond B7 27
Colab memory and scope 27
Interview questions 28
12. Sampling the next token (C1) 29
Scores are not the choice 29
Order and conventions 29
Precision, randomness and validation 29
Interview questions 30
13. The generation loop (C2) 31
Grow the context 31
Output and stopping contract 31
Raw text versus a chat template 31
Verification 32
Interview questions 32
14. Contiguous KV cache (C3) 33
Why the old K/V remain valid 33
Shapes and memory 33
Prefill, then decode 33
Rectangular causal mask 33
Integration and boundaries 34
Verification and current status 34
Interview questions 35
15. No-cache versus cached speed curve (C4) 36

The question 36
What caching removes 36
Two controlled sweeps 36
Fixed-length workloads and EOS 36
Timing boundaries 37
Ranges and trial order 37
Memory 37
Evidence and status 37
Interview questions 38
16. Flattened batching and attention metadata (D1) 40
From one request to several 40
Three pieces of layout information 40
Why resetting positions is not enough 40
Packing is not padding 40
Output routing and next tokens 41
Why the model need not be rewritten 41
Boundaries with C3 and D2 41
Verification and status 41
Interview questions 42
17. Block tables, slot mapping and paged KV cache (D2) 43
Why change the storage layout? 43
Logical position versus physical location 43
Shared pool and per-request lengths 43
Readable paged attention 44
Commit, retry and reset 44
Validation and interface 44
Verification and status 45
Interview questions 45
18. Block ownership, reference counts and reuse (D3) 46
What changes after D2? 46
The allocator contract 46
Reservation is not committed length 46
Failure, reset and release 47
Memory and performance boundaries 47
Validation 47
Three interview questions 48
19. Continuous batching and recomputation preemption (D4) 49
What is the scheduler deciding? 49
Why continuous batching? 49
Three meanings of "length" 49
First worked example: a new arrival joins an older request 49
What if there are not enough blocks? 50
Second worked example: an actual preemption 50
Execution, failure and cancellation 51
Finite teaching workloads, not a serving API 51
Validation and status 51
Three interview questions 52
20. Engine loop and memory budgeting (D5) 53
From a scheduling decision to generated text 53
Processed tokens are not sampled outputs 53
Continuous arrivals and cancellation 53

Preemption keeps history, not random decisions 54
Failure is not success 54
KV memory: count KV heads, not query heads 55
Budgeting against a live GPU 55
Tests and notebook evidence 56
Recorded Colab validation 56
Three interview questions 57
21. Streaming chat over loopback HTTP (E1) 58
Status and scope 58
Where HTTP fits 58
A deliberately limited OpenAI-compatible subset 58
What streaming means 59
Why not decode each token independently? 59
One owner for the engine 59
Slow clients and retained histories 60
Disconnects, failures and shutdown 60
Tests and evidence 60
Three interview questions 61
22. Measure the engine against Hugging Face (E2) 62
Status and scope 62
Define the boundaries before measuring 62
Observe one full run 62
Keep work and memory explicit 63
Trials, correctness and conclusions 63
Recorded Colab result 64
Three interview questions 64
Stop here: review the core project 65
A. Start from tokens and hardware 65
B. Rebuild the model mentally 65
C. Generate without changing semantics 65
D. Own and schedule memory correctly 65
E. Serve and measure honestly 65
Evidence and limits 65

How to use this study guide

This is the core TinyServe curriculum: 22 chapters from tokens to a small inference server and direct benchmarks. Read in numeric order. Each chapter connects concepts, source code, tests, recorded examples and interview questions. The masterplan defines the full roadmap.

The chapters preserve the teaching progression: a chapter can describe a later lesson as future work even though that lesson is now complete. Current runtime evidence lives in the results index, not in those historical future-tense passages. Counts from different regression suites overlap; do not add them into a unique-test total.

Reading order

Part	Chapters
A: foundations	01 Tokens, 02 Roofline, 03 HF baseline
B: model	04 Weights, 05 RMSNorm, 06 RoPE, 07 Attention, 08 GQA, 09 MLP, 10 Decoder, 11 Full model
C: generation	12 Sampling, 13 Generation, 14 KV cache, 15 Cache timing
D: batching and paging	16 Flattening, 17 Paging, 18 Block manager, 19 Scheduler, 20 Engine
E: serving	21 Streaming server, 22 HF comparison

Use the core-review checklist after E2 and H. Pause there before Parts F/G. This review is a user-requested learning checkpoint, not a claim that later performance features already exist.

PDF and reproducible build

Read `docs/tinyserve-study-guide.pdf` and its build manifest, `docs/tinyserve-study-guide.manifest.json`, directly from the repository. Part H uses a local documentation-only build, not CI or distribution builds. There is no release or deployment. The PDF includes this introduction, all numbered chapters, the existing roofline image and the core-review checklist. Its table of contents is navigable. Code/result links point to the source commit recorded by the build.

Required documentation tools:

- Pandoc 3.13.
- Typst 0.15.1.

Use official release archives and verify their published SHA256 digests before extraction. The build records the link-target commit and source-file/PDF hashes in `tinyserve-study-guide.manifest.json`.

From a clean checkout, with both executables on PATH:

```
python scripts/check_static.py
python scripts/build_guide.py --revision FULL_40_CHARACTER_COMMIT_SHA
```

Replace the placeholder with `git rev-parse HEAD`. An explicit SHA fixes the code/result link targets; it does not prove a dirty tree matches that SHA. The manifest hashes identify the actual local document inputs. For an exact committed-source build, use a clean checkout or export the committed source first. Output defaults to `docs/tinyserve-study-guide.pdf`, alongside its manifest. `--pandoc`, `--typst` and `--output` accept explicit paths when tools are not on PATH.

The user approved local documentation-only builds for H. This does **not** permit local pytest, notebook/model execution, dependency setup or weight downloads. Runtime lessons remain Colab-only. Local static and PDF checks do not replace the archived T4 runs. No byte-for-byte cross-platform PDF reproducibility is promised: system fonts can differ.

01. Tokens, tensors, shapes, logits (Lesson A1)

The pipeline

`text --tokenizer--> ids (N,) --embedding--> (N, 2048) --22 decoder layers--> (N, 2048) --lm_head--> logits (N, 32000)`

Measured on TinyLlama-1.1B-Chat (T4, FP16), prompt `"The capital of France is"`:

Stage	Shape	Note
ids	`(6,)` int64	`<s> _The _capital _of _France _is`; `<s>` (id 1) is BOS, added by the tokenizer
after embedding	`(6, 2048)`	one 2048-vector per token
logits	`(6, 32000)` FP16	384,000 bytes = 6 x 32000 x 2

Model facts: 1,100,048,384 parameters, 2.20 GB in FP16, 22 layers, hidden 2048, 32 query heads, 4 KV heads, head_dim 64, vocab 32000.

Ideas to remember

1. **The model only sees integer ids.** The tokenizer maps text to ids and back. `_` marks a leading space.
2. **Flattened tokens.** We keep ids as one 1-D tensor `(N,)`, not `(batch, seq)`. Later, many requests will share one flat tensor plus metadata saying where each begins.
3. **Row `i` of the logits scores the token after position `i`.** For the prompt above the last row gives `_Paris` 50.9%, `_located` 16.5%, `:` 2.2%. Generation uses only the last row; the other rows are wasted work at inference (we will skip them in Part D).
4. **Greedy decoding** is `argmax` of the last row. Sampling (Part C) draws from the softmax instead.
5. **Naive generation** appends the token and re-runs the whole sequence each step. Correct, but every step redoes all earlier work.
6. **Causality.** The attention mask stops position `i` from seeing later tokens, so its logits do not change when tokens are appended. In FP32 on CPU the test shows differences below 1e-5. In FP16 on the T4 we measured a max difference of 0.069. That is most likely rounding noise (FP16 has about 3 decimal digits, logits are order 10, and differently shaped kernels accumulate in a different order across 22 layers), not a causality leak; the FP32 test is the strict check. This is why FP16 is compared against FP32 with a tolerance, not demanded bit-equal.

Code map

- basics.py: `load_model`, `encode`, `forward_logits`, `next_token_greedy`, `generate_greedy_naive`, `prefix_logit_diff`, `shape_report`.
- test_a1_basics.py: logic tests on a tiny random Llama (CPU) and one `slow` test on the real config.
- A1 notebook and results.

Interview questions

1. **What shape are the logits, and which row do you use to pick the next token?** `(N, vocab)`, one row per input position. Row `i` predicts the token after position `i`, so generation uses the last row. All earlier rows only matter for training or scoring.
2. **Why does appending tokens not change the logits of earlier positions, and why does that matter?** Causal masking: a position attends only to itself and earlier positions. So the keys and values of earlier tokens never change as the sequence grows, which is what makes caching them (the KV cache) valid.
3. **Why is the naive generation loop slow, and what does a decode step cost in memory traffic?** It recomputes all N positions at every step, so total work grows quadratically in the number of

generated tokens. Even with a cache, each decoded token must read all 2.2 GB of FP16 weights, which is why decode is memory-bandwidth-bound (Lesson A2).

02. GPU basics: bandwidth, matmul, roofline (Lesson A2)

Three ideas

1. **Decode is limited by memory bandwidth.** Producing one token reads every weight once.
2. **Prefill is limited by compute.** Many tokens share one read of the weights.
3. **Roofline.** A kernel does `FLOPs` of work and moves `bytes`. Its *arithmetic intensity* is `FLOPs / bytes`. The *ridge point* is `peak FLOPs / peak bandwidth`. Below it the kernel is memory-bound (speed = intensity x bandwidth); above it, compute-bound (speed = peak FLOPs).

For a decode matmul `(B, K) @ (K, N)` the weight (`K*N` elements) dominates the bytes, so intensity is about `B`. **Batching raises intensity at almost no extra memory cost.**

What we measured on the T4 (FP16, ranges over 7 trials)

Quantity	Datasheet	Measured	Note
Memory bandwidth (256 MiB copy)	320 GB/s	**241 GB/s** (240.5 to 241.2 at 64 MiB)	75% of datasheet. 1 MiB buffers are launch-overhead dominated, so we use 64 MiB and up
FP16 matmul, sustained (n=8192)	65 TFLOPS	**19.4 TFLOPS** (19.0 to 19.8)	30% of datasheet
FP16 matmul, burst (n=1024)	65 TFLOPS	37.3 TFLOPS	short run before the clock drops; n=2048 swung between 13.6 and 41.2
FP32 matmul	8.1 TFLOPS	3.0 TFLOPS	37% of datasheet

Footnote: `nvidia-smi` showed a 70 W power limit and an SM clock of 585 MHz against a 1590 MHz maximum at 72 C. That is why the T4 varies run to run and why we report ranges, and why we use the *sustained* number as the compute roof.

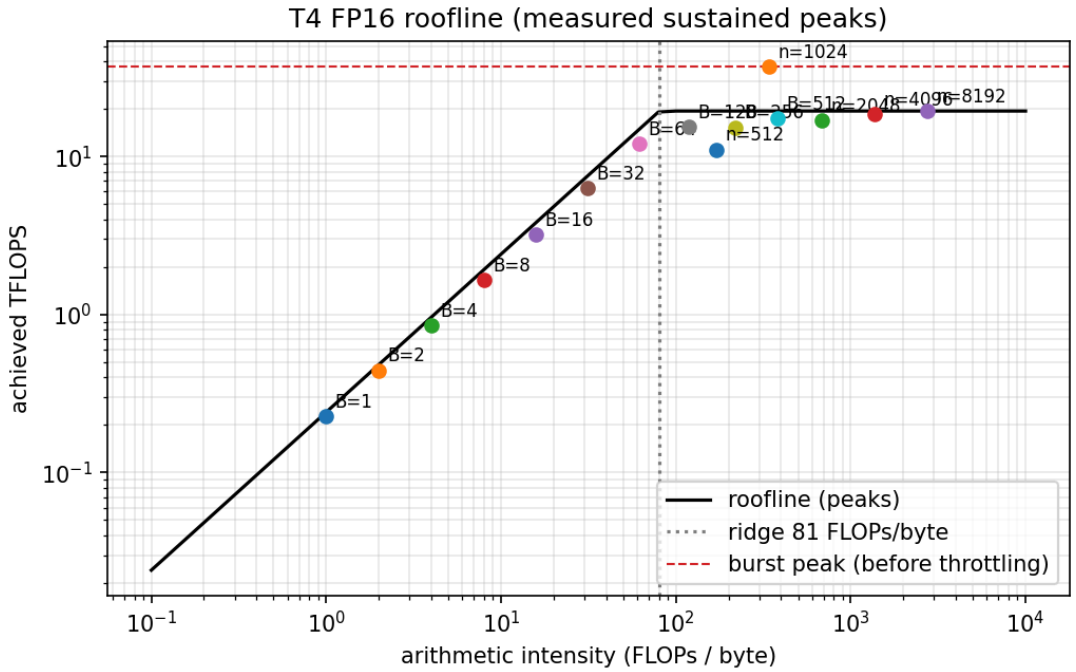
Ridge point: 65 TFLOPS / 320 GB/s = 203 FLOPs/byte on the datasheet, but 19.4 TFLOPS / 241 GB/s = **81 FLOPs/byte** on measured sustained numbers. We use the measured one.

Batching, measured

One TinyLlama MLP up-projection, `(B, 2048) @ (2048, 5632)` in FP16 (23 MB of weights, larger than L2):

B	ms	intensity	GB/s	TFLOPS	rows/s
1	0.102	1.0	227	0.23	9,827
8	0.111	8.0	209	1.66	71,983
64	0.123	61.4	196	12.0	522,057
128	0.191	117.9	131	15.5	669,736
512	0.681	381.8	45	17.4	752,217

From B=1 to B=64 the time grows by about 20% while the work grows 64x, so rows per second grow about 53x. Past B=64 the intensity crosses the ridge (81), time starts growing with `B`, and throughput flattens near the compute roof.



Decode ceiling for TinyLlama

`weight bytes / bandwidth` = 2.2 GB / 241 GB/s = about 9.1 ms per step, so at most **about 110 tokens/s at batch 1**. Real decode is slower (KV-cache reads, small-kernel overheads, sampling, Python). In Part E we will compare actual speed to this ceiling.

Caveat: batching amortizes the *weight* reads. Attention over the KV cache reads each sequence's own cache, so that part does not get cheaper per token with batching. Long contexts and large batches therefore eventually become KV-bandwidth-bound too.

Code map

- roofline.py: pure-Python math (FLOPs, bytes, intensity, ridge, attainable FLOPs, decode ceiling, `summarize` for ranges).
- gpu.py: CUDA-event timing, bandwidth / matmul / decode-curve measurements.
- plots.py: the roofline diagram as code.
- test_a2_roofline.py: exact math tests plus GPU sanity checks (9 pass on the T4).
- A2 notebook and results.

Interview questions

1. **Why is single-sequence decode memory-bound, and what does batching change?** Each step does about 2 FLOPs per weight but must read each 2-byte weight from memory, so intensity is about 1 FLOP/byte, far below the T4's ridge (about 81). Speed is set by bandwidth. Batching `B` sequences reuses one weight read for `B` tokens, so intensity rises to about `B` and throughput scales almost linearly until the ridge point (we measured about 53x for B=64).

2. **What is the roofline model, and how do you compute the ridge point?** Attainable speed = `min(peak FLOPs, intensity x peak bandwidth)`. The ridge point is peak FLOPs divided by peak bandwidth, the intensity where the two limits meet. Left of it a kernel is memory-bound, right of it compute-bound. On our T4: 19.4 TFLOPS / 241 GB/s = 81 FLOPs/byte.

3. **How would you estimate the best-case decode speed of a model, and why would real numbers be lower?** `bandwidth / bytes of weights read per step`: 241 GB/s / 2.2 GB = about

110 tokens/s for TinyLlama FP16 at batch 1. Real decode is lower because of KV-cache reads (which do not amortize with batching), launch overheads, sampling and Python. Also, measured numbers (241 GB/s, 19.4 TFLOPS) are well below datasheet peaks, so use measured ones and report ranges.

03. The Hugging Face `generate` baseline (Lesson A3)

Everything we build later is compared with these numbers. Setup: TinyLlama-1.1B, FP16, T4, greedy decoding, prompt 128 tokens, 128 new tokens, identical prompts across the batch, 5 trials per point (ranges in A3_hf_baseline.json).

Definitions (kept for our own engine)

- **TTFT**: time to first token = prefill + one token. Measured with a 1-token `generate`.
- **Step latency**: `(time for N tokens - TTFT) / (N - 1)`. Every sequence in the batch gets one token per step, so this is also the inter-token latency.
- **Decode tokens/s** = `batch / step latency`.
- **MBU**: `steps/s x weight bytes / measured bandwidth` (A2: 2.2 GB, 241 GB/s).

Results

Batch	TTFT (ms)	Step latency (ms), min / median / max	Decode tok/s	A2 ceiling tok/s	MBU	Peak GB
1	39.7	30.71 / 30.90 / 31.15	32	110	30%	2.22
2	44.3	30.97 / 31.54 / 32.00	63	219	29%	2.24
4	76.6	31.80 / 32.42 / 35.01	123	438	28%	2.27
8	163.3	31.66 / 31.93 / 32.01	251	876	29%	2.32
16	331.2	33.37 / 33.75 / 33.99	474	1753	27%	2.44
32	694.2	52.01 / 52.10 / 52.16	614	3506	18%	2.67

Prefill at batch 1 (TTFT min / median / max): 32 tokens 36.5 / 41.5 / 55.1 ms, 128 tokens 37.6 / 38.3 / 41.4 ms, 512 tokens 119.7 / 120.9 / 121.4 ms, 1024 tokens 324.5 / 326.2 / 328.7 ms. Estimated throughput (2 x 969M matmul parameters x tokens / TTFT) is only 1.5, 6.5, 8.2 and 6.1 TFLOPS, against the roughly 19.4 TFLOPS sustained roof from A2.

With `use_cache=False` at batch 1 the step latency is 42.9 ms instead of 30.9 ms (1.4x slower, only 2 trials).

What the numbers say

1. **Batch-1 decode reaches 32 tokens/s, about 30% of the 110 tokens/s bandwidth ceiling.** The GPU is not the limit.
2. **The step latency stays almost flat (30.9 to 33.8 ms) from batch 1 to 16,** so throughput scales almost linearly (32 to 474 tokens/s). A step that costs the same whether it moves 2.2 GB for 1 sequence or 16 sequences is mostly a fixed cost. The likely cause is per-step overhead from Python and the many small kernel launches across 22 layers, which is consistent with the data but not proven here (we have not profiled). We will test it in Part F (CUDA graphs).
3. **At batch 32 the step jumps to 52 ms** (MBU 18%); something that does scale with batch (attention over more KV cache, more compute) starts to matter. We will see in Part F.
4. **The KV cache helps only modestly here (1.4x)** because the context is short (128 to 256 tokens) and the cached step is overhead-bound. The cache saves work that grows with context length, so the gap widens for long sequences. We will measure that curve in C4.
5. **TTFT grows roughly linearly with batch x prompt tokens** (batch 8: 163 ms, batch 32: 694 ms). Larger batches improve throughput but make the first token wait: latency and throughput trade off.
6. **Prefill is well below the compute roof** (6 to 8 TFLOPS against about 19). The cause is unknown; short prompts are dominated by the roughly 37 ms floor. This is an open question for our own implementation.

Targets for our engine

- Batch 1 decode: beat **32 tokens/s** (30% MBU).

- Batch 32 decode: beat **614 tokens/s**.
- Remember HF's static batching: all rows share one length and the batch finishes when the slowest does. Continuous batching (Part D) removes that.

Code map

- hf_baseline.py: `make_prompt_ids`, `timed_generate`, `measure_ttft_ms`, `run_case`.
- roofline.py: `model_bandwidth_utilization` added.
- test_a3_bench.py: tiny-model tests (exact output length, fields, ordering); 16 tests pass with the A2 tests.
- A3 notebook.

Interview questions

1. **Define TTFT, inter-token latency and throughput, and how do you measure TTFT separately from decode?** TTFT is the wait until the first token (prefill plus one token). Inter-token latency is the time between later tokens. Throughput is total tokens per second across all requests. Measure TTFT with a 1-token generation; get step latency from `(N-token time - TTFT) / (N - 1)`, with a device sync around the timed region.

2. **HF decode runs at 30% of the bandwidth ceiling and its step time barely changes from batch 1 to 16. What does that tell you?** The step is dominated by a cost that does not depend on batch size, likely CPU and launch overhead from many small kernels, so the GPU is idle part of the time. More batching is nearly free throughput until the real limits appear (batch 32 here). The fixes are fewer and fused kernels and CUDA graphs, which replay a whole step with one launch.

3. **Why does a bigger batch raise throughput but hurt latency, and what does static batching add to the problem?** Weights are read once per step for the whole batch, so tokens per second grows, but a larger batch means more prefill work before anyone's first token (TTFT 40 ms at batch 1 vs 694 ms at batch 32). With static batching all sequences run until the longest finishes and new requests wait for the next batch; continuous batching admits and retires requests at every step.

04. Weight tour (Lesson B1)

Before writing model code we looked at what is inside the checkpoint. Our own model must load these tensors by name, so weights.py holds a pure-Python spec of every name and shape, checked against the real model on the T4 (results: B1_weights.json).

The 201 tensors

`1 (embedding) + 9 per layer x 22 layers + 1 (final norm) + 1 (lm_head) = 201`, all FP16. One decoder layer holds:

Tensor	Shape	Parameters	Role
`self_attn.q_proj`	(2048, 2048)	4,194,304	32 query heads x 64
`self_attn.k_proj`	(256, 2048)	524,288	4 KV heads x 64
`self_attn.v_proj`	(256, 2048)	524,288	4 KV heads x 64
`self_attn.o_proj`	(2048, 2048)	4,194,304	mixes the heads back to 2048
`mlp.gate_proj`	(5632, 2048)	11,534,336	SwiGLU gate
`mlp.up_proj`	(5632, 2048)	11,534,336	SwiGLU up
`mlp.down_proj`	(2048, 5632)	11,534,336	back to 2048
`input_layernorm`	(2048,)	2,048	RMSNorm before attention
`post_attention_layernorm`	(2048,)	2,048	RMSNorm before the MLP

Total per layer **44,044,288**; x 22 = 968,974,336. Add the embedding table and `lm_head` (32000 x 2048 = 65,536,000 each, not tied) and the final norm: **1,100,048,384** parameters, exactly what the spec predicts from the config alone.

PyTorch `nn.Linear` stores weights as `(out_features, in_features)`, which is why `k_proj` is `(256, 2048)`.

Where the bytes are

Part	Parameters	Bytes	Share
MLP	761.3 M	1522.5 MB	**69.2%**
Attention	207.6 M	415.2 MB	18.9%
Embedding	65.5 M	131.1 MB	6.0%
lm_head	65.5 M	131.1 MB	6.0%
Norms	0.1 M	0.2 MB	0.0%

Every decode step reads each weight once (A2), so the MLP alone is about 69% of the weight traffic. One refinement: the embedding table is only a lookup (one row per token), so a decode step reads about 2.07 GB, not 2.20 GB. That moves the batch-1 bandwidth ceiling from 110 to about 116 tokens/s and the measured HF MBU from 30% to about 28%. The A2 and A3 chapters used the full 2.2 GB, which is a slight overestimate of the bytes read.

GQA and the KV cache

Query heads 32, KV heads 4: 8 queries share each KV head. So `k_proj` and `v_proj` have 256 outputs instead of 2048, and the cache stores only 4 heads per layer:

```
KV bytes per token = 2 x 22 layers x 4 KV heads x 64 x 2 bytes = 22,528 bytes = 22 KiB
```

Full multi-head attention would need 180,224 bytes (176 KiB) per token, 8x more. Consequences:

- One 2048-token sequence costs 44 MiB of cache, versus 2098 MiB for the weights.
- With 10 GiB free, about 476,000 tokens fit: **232 sequences of 2048 tokens**. This is the budget Part D's block manager has to manage.

What the weights look like (FP32 statistics)

Tensor	mean	std	min	max
embed_tokens	−0.0000	0.0149	−0.117	0.149
layer 0 q_proj	−0.0000	0.0164	−1.586	1.031
layer 0 down_proj	0.0000	0.0166	−0.523	0.414
layer 0 input_layernorm	0.0058	0.0460	−0.582	0.770
layer 21 input_layernorm	0.4637	0.0852	0.114	1.227
final norm	1.9149	0.1365	0.508	3.188
lm_head	−0.0004	0.0247	−0.471	0.395

Projection weights are tiny and centred on zero, but have rare outliers far from the typical size (q_proj max magnitude 1.586 against std 0.016, about 100 sigma). Outliers like these matter when we quantize (Part G). Norm scales differ a lot between layers: the first layer's `input_layernorm` is close to zero on average while the last is about 0.46 and the final norm about 1.9. We observe this; we do not yet explain it.

Code map

- weights.py: `expected_shapes`, `expected_param_count`, `kv_cache_bytes_per_token`, `compare_shapes`, `weight_tour`, `bytes_by_category`, `tensor_stats`.
- test_b1_weights.py: exact arithmetic tests, a state_dict check on a tiny Llama, and the real config check. 13 tests pass with the quiet-logs tests.
- B1 notebook.

Interview questions

1. **Where do TinyLlama's 1.1B parameters live?** About 969M (88%) are in 22 identical decoder layers of 44M each: the MLP (three 2048 x 5632 matrices) is 69% of all bytes and attention 19%. The rest is the embedding and the separate `lm_head` (65.5M each) and tiny norm vectors.
2. **What is grouped-query attention and what does it save?** Many query heads share fewer KV heads (32 queries, 4 KV heads). The K and V projections shrink 8x, and the KV cache is 8x smaller: 22 KiB per token instead of 176 KiB. Quality is close to full multi-head attention, while memory and bandwidth for the cache drop a lot, so more sequences fit in a batch.
3. **How do you budget GPU memory for serving?** Weights are fixed (2.1 GiB here); the rest, minus activations and overhead, is for the KV cache at 22 KiB per token. With 10 GiB free that is about 476,000 tokens, e.g. 232 full 2048-token sequences. The block manager (D3) divides this into fixed-size blocks so no sequence reserves more than it uses.

05. RMSNorm (Lesson B2)

Code: norm.py. Tests: test_b2_rmsnorm.py (13 pass). Probe helpers: probe.py. Results: B2_rmsnorm.json.

The operation

```
y = x / sqrt(mean(x^2) + eps) * weight        mean over the 2048 hidden values of ONE token
```

Input and output are `(N, 2048)`; every token is normalised independently. TinyLlama has 45 of them: an input norm and a post-attention norm in each of the 22 layers, plus a final norm. `eps = 1e-5`.

Hand example: `x = [3, 4]`, `mean(x^2) = 12.5`, `rms = 3.5355`, `y = [0.8485, 1.1314]`.

RMSNorm vs LayerNorm. LayerNorm computes the mean and the variance and subtracts the mean, then adds a bias; RMSNorm skips the mean subtraction and the bias, so it needs one reduction instead of two. The difference is visible: adding 5 to every input leaves LayerNorm's output unchanged (`[1.47, -0.08, ...]`), but changes RMSNorm's (`[1.28, -0.24, ...]` becomes `[1.32, 0.95, ...]`).

Implementation details that matter

1. **Statistics in FP32.** FP16 tops out at 65504, so squaring any value above 255.9 overflows. An all-FP16 version of `x * rsqrt(mean(x^2) + eps)` on `x = 300` gives `mean = inf`, `rsqrt(inf) = 0`, and the output is **all zeros with no error**. Ours squares in FP32 and returns `1.0` (measured on the T4).
2. **Order of casts.** We normalise in FP32, cast back to the input dtype, and only then multiply by the weight, exactly as Hugging Face does. That makes FP16 outputs identical to HF, not merely close.
3. **Shape-agnostic, dtype-preserving.** Works on `(N, hidden)` or `(batch, seq, hidden)` and returns the input dtype. The module takes `device` and `dtype` arguments (PyTorch convention) and its parameter is named `weight`, matching the checkpoint (B1).

Test results

Check	Tolerance	Why
vs float64 formula (FP32)	atol 1e-6	FP32 rounding only
vs HF `LlamaRMSNorm` (FP32)	1e-6	same operations
vs HF (FP16)	exact equality	same operation order
FP16 vs FP32 computation	rtol 2e-3, atol 1e-3	two FP16 roundings, each at most 2^-11 relative
scale invariance (`eps=0`)	atol 1e-5	`norm(3x) == norm(x)`
FP16 overflow	all zeros (naive), 1.0 (ours)	the trap above

Real model. We recorded the input and output of all 45 norms during one forward pass of "The capital of France is" (FP16, T4) and fed each real input to our module with the same weight: **45 of 45 outputs exactly equal, worst difference 0.0**.

What the real activations look like

The residual-stream values entering the norms (one 6-token prompt, so treat this as an observation, not a general rule):

Where	Input RMS	Input max abs
layers 0 to 2	0.010 to 0.028	0.1 to 0.9
layers 3 to 7	1.589	**140.6**
layers 8 to 17	1.58	139.5 to 138.8
layers 18 to 21	1.59 to 1.62	138.4 to 134.1
final norm	0.727	20.7

- The RMS jumps from 0.028 to 1.589 between layer 2's post-attention norm and layer 3's input norm, so layer 2's MLP writes a very large value into the residual stream; from then on the maximum stays at about 140 until the last layers.
- That single value makes up most of the energy: the squared RMS over the `6 x 2048` input elements gives a total of about 31,000, and `140.6^2 = 19,775`, about 64% of it. The largest element is about 88 times the RMS.
- This looks like the "massive activations" effect reported for large language models. We did not check which token and dimension it is in, so that link is a hypothesis.
- For FP16: `140.6^2 = 19,775` is below 65,504, but only by a factor of 3.3. Computing the statistic in FP32 is not a theoretical nicety here.

Code map

- norm.py: `rms_norm`, `RMSNorm`.
- probe.py: `capture_module_io` (hooks) and `compare_rmsnorm` (our module vs HF on real activations). We will reuse this for RoPE, attention, the MLP and the full model.
- B2 notebook.

Interview questions

1. **How does RMSNorm differ from LayerNorm, and why do Llama models use it?** RMSNorm divides by the root-mean-square only: no mean subtraction and no bias. It needs one reduction instead of two and has fewer parameters, and in practice trains as well as LayerNorm. A consequence is that it is not invariant to adding a constant to the input, while LayerNorm is.

2. **Why is the RMSNorm statistic computed in FP32 when the model runs in FP16?** Squaring overflows FP16 above 255.9 (65504 maximum), and one overflow turns the mean into `inf` and the output into zeros without any error. Real TinyLlama activations reach about 140 (`140^2 = 19,775`, only 3.3x from the limit). We upcast the statistic, then cast back to the input dtype before applying the weight.

3. **How did you verify your RMSNorm is correct?** Four levels: a hand-computed example, a float64 formula, the Hugging Face `LlamaRMSNorm` (tight 1e-6 tolerance in FP32, exact equality in FP16 because the operation order matches), and all 45 norms of the real model fed with real captured activations (45 of 45 exactly equal). FP16 versus FP32 is checked with a tolerance derived from FP16 rounding rather than demanded bit-equal.

06. Rotary position embeddings (B3)

Code: rope.py. Tests: test_b3_rope.py. Notebook: B3_rope.ipynb. Recorded results: B3_rope.json.

Give attention position information

A query/key dot product does not know token positions by itself. RoPE rotates queries and keys by position-dependent angles. It does not rotate values or add a position vector to the embedding.

TinyLlama uses head dimension 64. Hugging Face's Llama convention pairs dimension `i` with `i + 32`, not adjacent dimensions. There are 32 pairs.

For pair `i`, position `p`, head dimension `D`, and base `theta`:

```
inv_freq[i] = 1 / theta ** (2 * i / D)
angle[i] = p * inv_freq[i]

rotate_half([first_half, second_half]) = [-second_half, first_half]
rotated = x * cos(angle) + rotate_half(x) * sin(angle)
```

Duplicate the 32 angles to obtain 64 cosine/sine entries. Both dimensions of each pair use the same angle. A two-dimensional rotation preserves vector length; position zero is the identity.

Hand example

For `D=4`, `theta=10000`, the two pairs are `(0, 2)` and `(1, 3)`. Their speeds are 1 and 0.01 radians per position.

```
x at position 1 = [1, 0, 0, 0]
pair (0, 2)    = [1, 0]
after rotation = [cos(1), sin(1)]
full output    = [0.5403, 0, 0.8415, 0]
```

Using adjacent pairs would give a different convention and would not match these checkpoint weights.

Fast and slow pairs

With TinyLlama's `D=64` and `theta=10000`:

Pair	Radians per position	Positions per full turn
0	1	6.283
16	0.01	628.319
31	approximately 0.000133352	approximately 47,117.24

Fast pairs resolve short distances. Slow pairs change gradually across the 2,048-token context. The model combines all frequencies; a fast pair's full turn is not a full-vector repetition.

Why the score depends on relative position

For a rotation matrix `R`, with position-scaled angles:

```
dot(R_m q, R_n k) = q.T @ R_m.T @ R_n @ k
                  = q.T @ R_(n-m) @ k
```

The positional part depends on `n-m`, not the absolute locations. Actual queries and keys still depend on token content.

The notebook repeats one fixed query and one fixed key at many positions. The score matrix should be constant along diagonals of equal offset. FP32 angle and arithmetic rounding introduces small differences.

Shapes, dtype and cache implications

Inputs use flattened layout:

```
positions: (N,)
queries:   (N, 32, 64)
keys:      (N, 4, 64)
cos/sin:   (N, 64), broadcast across heads
```

Positions are explicit. Packed sequences restart their own positions; cached decode continues from the existing prefix length. Angles and trigonometry use FP32, then cosine/sine are cast to the input dtype, matching HF's recipe. `get_rope_theta` supports both the older configuration field and Transformers 5's `rope_parameters`.

C3 stores keys after this rotation. Reuse each old key at its original position without rotating it again. Values stay unrotated.

Recorded validation

The saved October 31, 2026 Colab notebook records **16 passed tests**. Tests cover hand values, identity, norm preservation, relative scores, packed/large positions, HF FP32/FP16 comparisons and configuration lookup.

Real TinyLlama comparisons use layers 0, 10 and 21, with ordinary and packed positions: six comparisons total. Cosine/sine, rotated queries and rotated keys matched HF exactly in every recorded comparison.

The fixed-vector diagonal spread was `1.7166e-5`, with query/key norm product approximately `69.2468`. The test bounds spread by `1e-4` times that scale to account for FP32 angle rounding.

Synthetic FP32 HF comparisons use absolute `1e-6`. Same-recipe FP16 comparisons check exact equality on the tested inputs; this is not a universal FP16 bit-equality requirement. A separate FP16-versus-FP32 check uses the same rounded inputs and absolute `2e-2`.

The archived JSON preserves the original T4 environment, timestamp and git SHA `335035c`. Its payload contains numerical comparisons, not pytest stdout; the test count remains in the saved notebook output.

Interview questions

1. **Why rotate queries and keys, but not values?** Position changes the similarity used for attention weights; values carry the mixed content.
2. **Why does RoPE encode relative position?** The two rotations combine into a rotation by their position difference in the dot product.
3. **What can break checkpoint parity?** Wrong pairing, frequency base, positions or dtype recipe. Cached keys must not be rotated twice.

07. Attention scores, causal mask and softmax (B4)

Code: attention.py. Tests: test_b4_attention.py. Notebook: B4_attention.ipynb.

The four steps

Each token supplies a query (what it seeks), a key (what it offers) and a value (the content to mix). These are useful analogies, not labels assigned to individual dimensions.

```
scores = Q @ K.T / sqrt(head_dim)
scores[not_allowed] = -inf
weights = softmax(scores, dim=keys)
output = weights @ V
```

One head with two tokens: queries `[1, 1]`, keys `[0, ln(3)]`, values `[10, 20]`, and head dimension 1.

Query	Allowed scores	Attention weights	Output
Token 0	`[0, -inf]`	`[1, 0]`	`10`
Token 1	`[0, ln(3)]`	`[0.25, 0.75]`	`17.5`

Softmax turns scores into nonnegative weights that sum to 1. PyTorch uses a stable implementation equivalent to subtracting the largest score before exponentiating. A masked score of `-inf` gets weight zero.

Why divide by sqrt(head_dim)?

If independent query/key components have roughly unit variance, a dot product of D components has variance about D. Dividing by sqrt(D) keeps the variance near 1 rather than letting larger heads make softmax overly peaked. TinyLlama has D=64, so the divisor is 8.

Shapes and causality

- Query: `(Nq, H, D)`.
- Key/value: `(Nk, H, D)`.
- Allowed mask: `(Nq, Nk)`, boolean; **True means visible**.
- Scores/weights: `(H, Nq, Nk)`.
- Output: `(Nq, H, D)`.

For one prompt, token i sees positions <= i, including itself. This is safe: its output predicts token i+1, not token i.

For packed requests, visibility also requires equal sequence IDs. Restarting positions alone does not isolate requests. The mask combines `key_position <= query_position` with `query_sequence == key_sequence`.

For single-token cached decode, one query can see all valid cached keys, including its own newly written key. Unused cache slots and other requests must still be excluded. B4's rectangular mask supports this shape, but B4 does not implement a cache.

Changing future Q/K/V must not change any earlier outputs. This is the property that makes old KV entries reusable.

Precision and validation

The reference uses FP32 for FP16/BF16 scores, softmax and value mixing, then casts the output back. FP64 inputs remain FP64. This is a deliberately readable correctness backend, not a fused production kernel. FP16 dot products can overflow even when all input components are finite.

Check	Tolerance	Reason
Hand example	absolute 1e-6	Small FP32 rounding
FP32 vs independent float64 formula	relative 1e-5, absolute 1e-6	Dot-product and softmax rounding
PyTorch MHA/SDPA	relative 1e-5, absolute 1e-6	Different reduction orders
Real HF eager attention, FP32	relative/absolute 1e-5	Same input projections and RoPE
FP16 vs FP32 on the same quantized inputs	relative/absolute 5e-4	One final FP16 rounding
BF16 vs FP32 on the same quantized inputs	relative/absolute 4e-3	One final BF16 rounding

Other tests cover zero future weights, row sums, packed vs separate requests, one-query decode, RoPE integration and invalid shapes/masks. A query with no visible keys raises an error instead of returning NaNs. Input checks synchronize the GPU; that is acceptable for this slow reference.

The real-model probe compares the input to HF's `o_proj` with our flattened head outputs for layers 0, 10 and 21. It uses HF's `repeat_kv` to expand TinyLlama's 4 KV heads to 32 query heads. **Our own GQA is B5**, not B4.

Run the notebook top to bottom on a Colab T4. It runs tests before demos, checks the real model, draws a heatmap, and writes environment information and measured differences to `results/B4_attention.json`. No results or test counts are claimed until that run succeeds.

Recorded Colab run (2026-10-31)

The saved B4 notebook contains **30 passed, none skipped** on a T4. The hand outputs were `[10, 17.5]`; future weights, changed-prefix difference and packed-vs-separate difference were all zero. FP16 vs FP32 differed by at most `0.000468612`. Real HF layer differences ranged from `9.31e-10` to `4.47e-8`, inside the FP32 tolerance.

B5 subsequently replaces the probe's borrowed HF head expansion with our own GQA. The B5 notebook reruns B4 tests to check that integration.

Interview questions

1. **Why scale attention scores?** Dot-product variance grows with head dimension; dividing by sqrt(D) controls the scale entering softmax.
2. **Why is the causal mask applied before softmax?** Disallowed scores become `-inf`, so their probabilities are zero and the remaining weights still sum to 1.
3. **How do you know attention is correct?** Check hand math, an independent float64 formula, PyTorch attention, causality and request isolation, then compare real HF activations in FP32. Compare FP16 to FP32 with a justified rounding tolerance rather than requiring bit equality.

08. Grouped-query attention (B5)

Code: gqa.py. Tests: test_b5_gqa.py. Notebook: B5_gqa.ipynb.

Share K/V, not queries or attention weights

TinyLlama has 32 query heads and 4 KV heads. Each consecutive group of 8 query heads uses one KV head:

```
query heads  0..7  -> KV head 0
query heads  8..15 -> KV head 1
query heads 16..23 -> KV head 2
query heads 24..31 -> KV head 3
```

Each query is different, so scores, probabilities and outputs can differ even within a group. Sharing a reference book does not mean asking the same question or getting the same answer.

Variant	KV heads with 32 query heads	Sharing
MHA	32	One KV head per query head
GQA (TinyLlama)	4	Eight query heads per KV head
MQA	1	All query heads share one KV head

Implementation and shapes

Input Q is `(Nq, 32, 64)`; K/V are `(Nk, 4, 64)`. `expand_kv_heads` repeats each KV head consecutively to `(Nk, 32, 64)`, then `grouped_query_attention` reuses B4's math. Output remains `(Nq, 32, 64)`, or `(Nq, 2048)` when flattened for the output projection.

`repeat_interleave(..., dim=1)` gives `[0,0,...,1,1,...]`. A cyclic repetition `[0,1,2,3,0,1,2,3,...]` pairs the wrong heads with the checkpoint's learned queries.

The query head count must be a positive multiple of the KV head count. K/V must match before expansion, not just after it. B4 retains the shape, dtype, device and mask validation. Equal head counts need no expansion.

Where the memory saving comes from

FP16 KV storage per token is:

```
2 (K and V) * 22 layers * 4 KV heads * 64 dimensions * 2 bytes = 22 KiB
```

Using 32 KV heads would cost 176 KiB: eight times as much. At 2048 tokens this is 44 MiB versus 352 MiB per sequence.

The reference **temporarily materializes expanded K/V** to reuse B4. The cache must store the original 4-head tensors. A later optimized backend reads shared KV heads directly without expanded copies.

GQA also shrinks K/V projection weights, but Q and O projections keep their original sizes. It does not reduce the number of query attention scores eightfold: all 32 queries still compare with the visible keys.

Verification

- Exact head mapping for 32/4, plus HF `repeat_kv` parity.
- MHA, GQA and MQA compared with separate per-query-head FP64 runs.
- Changing one KV head affects only its corresponding query group.
- Shared KV can produce different attention weights for different queries.
- Packed request isolation and cached-decode shape checks.
- FP16/BF16 compared with FP32 on the same quantized inputs, with B4's final-rounding tolerances (`5e-4` and `4e-3`).

- Real TinyLlama layers 0, 10 and 21 compared with FP32 HF eager attention, `rtol=atol=1e-5`. The probe now uses our GQA, not HF expansion.

The notebook runs B4 and B5 tests together to protect the shared probe. Recorded results are archived in B5_gqa.json.

Recorded Colab run (2026-10-31)
B4/B5 together passed **58 tests, none skipped** on a T4. The consecutive head mapping matched; shared-KV hand outputs were `18.80797` and `11.19203`. Future weights were zero. FP16 vs FP32 maximum difference was `0.000967979`, inside the combined relative/absolute assertion. Real HF FP32 differences ranged from `9.31e-10` to `4.47e-8`.

Interview questions

1. **How do MHA, GQA and MQA differ?** They differ in how many query heads share a KV head: one, a group, or all of them.
2. **Why does GQA shrink the cache without shrinking the output?** Cache storage follows KV head count; output follows query head count.
3. **Does this implementation avoid all expanded KV memory?** No. It is a readable reference with temporary copies. Persistent cache storage stays compact; a later kernel avoids the temporary expansion.

09. Llama MLP and SwiGLU (B6)

Code: mlp.py. Tests: test_b6_mlp.py. Notebook: B6_mlp.ipynb.

One token at a time

Attention mixes information across tokens. The MLP transforms each token independently, with the same learned weights applied to every row.

```
gate = gate_proj(x)                  2048 -> 5632
up = up_proj(x)                      2048 -> 5632
mixed = silu(gate) * up              elementwise
output = down_proj(mixed)            5632 -> 2048
```

`silu(z) = z * sigmoid(z)`. This is SwiGLU: a SiLU-activated branch modulates another branch. The gate is not a probability mask; it can be negative and can exceed 1. There is no softmax or causal mask here.

An ordinary two-projection feed-forward network has one up projection, an activation, and one down projection. SwiGLU has two up-sized branches and one down projection.

Hand example

Let `x=[1,-1]`, gate weights be identity, up weights be twice identity, and down weights be identity. Then:

```
gate = [1, -1]
silu(gate) = [0.731059, -0.268941]
up = [2, -2]
output = [1.462117, 0.537883]
```

The second result is positive because two negative numbers multiply.

Weights and shapes

PyTorch stores linear weights as `(output_features, input_features)`:

Name	TinyLlama shape	Parameters per layer
gate_proj.weight	`(5632, 2048)`	11,534,336
up_proj.weight	`(5632, 2048)`	11,534,336
down_proj.weight	`(2048, 5632)`	11,534,336

Total: **34,603,008 per layer**, **761,266,176 across 22 layers**, about 69.2% of TinyLlama's parameters. With uniform FP16 storage that is also 69.2% of its weight bytes, not necessarily 69.2% of execution time.

The module names match HF checkpoint keys and uses strict state-dict loading. There are no biases. Input/output preserve leading dimensions, device and dtype; `(N, 2048)` is the engine's flattened-token convention.

Verification and numerics

- Hand example and an explicit float64 formula.
- FP32 vs HF `LlamaMLP`, with the same weights.
- Zero gate/up branches, independent rows and strict loading errors.
- FP16/BF16 vs FP32 using the same rounded inputs and weights.
- All 22 real HF MLP inputs fed to our module in FP32; tolerance `rtol=atol=1e-5`, enforced by the probe.

The module uses the same operation order as HF, without special FP32 upcasting in its forward pass. Unlike B4's reference attention, multiple intermediate results are rounded in FP16.

Small synthetic FP16 tests use `rtol=atol=5e-3`, BF16 `4e-2`: several intermediate roundings plus cancellation near zero require both relative and absolute allowances. These are bounds for the controlled test inputs, not universal error guarantees. Real-model parity uses FP32 and tighter checks instead. Do not loosen a failed tolerance without inspecting why.

The notebook saves test output, environment metadata and the range of differences across all tested layers to `results/B6_mlp.json`. This is a correctness lesson, not a speed benchmark.

Recorded Colab run (2026-10-31)
All **74 B4-B6 tests passed**, none skipped. All 22 real MLP outputs matched HF exactly in FP32. Changing the final synthetic token changed earlier outputs by zero; the small FP16 comparison differed by at most `0.000224844`. These results come from the saved B6 notebook at commit `cc01a3f`. The original payload is archived in B6_mlp.json.

Interview questions

1. **What is SwiGLU?** `down(silu(gate(x)) * up(x))`, with separate learned gate/up branches and an elementwise product.
2. **Does an MLP mix tokens?** No. It mixes features within each token; attention handles cross-token interaction.
3. **Why does the MLP hold most of TinyLlama's weights?** Three large hidden-to-intermediate-sized matrices per layer, versus smaller attention projections, especially K/V under GQA.

10. Decoder layer and residuals (B7)

Code: llama.py. Tests: test_b7_decoder.py. Notebook: B7_decoder_layer.ipynb.

Connect the pieces

```
attention_delta = attention(input_layernorm(x))
after_attention = x + attention_delta
mlp_delta = mlp(post_attention_layernorm(after_attention))
output = after_attention + mlp_delta
```

Both branches read a normalized input. Both residual additions preserve the **unnormalized** stream. This is pre-norm, not norm-after-add (post-norm). Neither branch changes the width: input/output are `(N, 2048)`.

Think of a draft and two edits: keep the draft, add the attention edit, then add the MLP edit to the updated draft. Do not replace the draft with an edit.

One layer contains two RMSNorms, one attention block and one MLP, with two residual additions. TinyLlama has 22 such layers. The final RMSNorm, embedding table, LM head and full stack belong to B8, not inside an individual layer.

Attention branch

Q/K/V projections -> RoPE on Q/K -> grouped-query attention -> O projection. Q has 32 heads, K/V have 4, each dimension 64. Concatenate the 32 head outputs back to 2048 features, then apply `o_proj`.

The module reuses B2-B6 operations without duplicating their math. Weight keys match an HF decoder layer; strict loading rejects missing or extra weights. This implementation supports the bias-free SiLU Llama shape used by TinyLlama, and no dropout or KV cache yet.

Flattened metadata

`AttentionMetadata` carries local positions and optional request IDs. Ordinary prefill uses positions `0..N-1`. Packed requests restart their positions and have distinct IDs, so attention cannot cross request boundaries. The metadata constructs B4's boolean mask. It is the initial no-cache layout; cache/block fields will arrive with C/D. B7 does not implement paged storage.

Verification

- Zero both output branches: the decoder is exactly the identity.
- Explicit two-step formula verifies norm/residual order.
- Zero attention alone: MLP adds to the original unnormalized residual.
- Future-token changes cannot change earlier outputs.
- Packed requests match separate runs.
- Checkpoint keys and invalid layouts/configurations are checked.
- Synthetic HF parity covers MHA, GQA and MQA.
- Real-model probe feeds each of 22 real HF layer inputs to our layer. It checks **isolated layers**, not accumulated full-stack error.

FP32 HF assertions use relative/absolute `1e-5`. A controlled small CUDA FP16-vs-FP32 test uses relative/absolute `8e-3` to allow rounding across norms, projections, activation/products and residual additions. This is not a universal error bound. B8 will test the complete stack separately.

The notebook runs B4-B7 tests in Colab before demos and saves test output, environment info and per-layer differences to `results/B7_decoder_layer.json`. The original payload is archived in B7_decoder_layer.json.

Recorded Colab run (2026-10-31)

All **94 B4-B7 tests passed**, none skipped. Identity, explicit wiring and prefix-causality differences were zero. Packed-vs-separate difference was `1.19e-7`. Real isolated layer differences ranged from `3.73e-9` to `9.54e-7`, within the FP32 assertions. Results used commit `451a831`.

Interview questions

1. **What is a residual connection?** Add a branch's output to its input stream instead of replacing that stream. It preserves a direct path for information and gradients.
2. **Pre-norm versus post-norm?** Pre-norm normalizes inputs to the branch before computation; post-norm normalizes after the residual addition.
3. **Does isolated layer parity prove full-model parity?** No. It verifies each layer on HF's inputs. B8 checks our own outputs feeding subsequent layers, final normalization and logits.

11. Full model, loading and FP32 parity (B8)

Code: llama.py. Tests: test_b8_model.py. Notebook: B8_full_model.ipynb.

Complete path

```
input IDs (N,)
 -> embedding lookup (N, 2048)
 -> 22 decoder layers (N, 2048)
 -> final RMSNorm (N, 2048)
 -> lm_head (N, 32000)
```

Each logits row predicts the token after that position. Logits are scores, not probabilities; greedy selection takes the largest score. TinyLlama uses separate embedding and LM-head weights. Tied embeddings are also supported and tested: the two modules then share one parameter.

The model forward requires `input_ids`, explicit `positions`, and an `AttentionMetadata` object. The positions must agree with the metadata's B7 position field. Packed requests restart positions and provide request IDs. Nothing in the model forward calls HF or its attention routines.

Weight loading

The hierarchy matches checkpoint keys: `model.embed_tokens`, `model.layers`, `model.norm` and `lm_head`. `from_huggingface` creates our architecture on the source device/dtype and copies the state dictionary with `strict=True`. Missing, unexpected or incorrectly shaped weights raise errors.

HF is the weight/tokenizer source and test oracle, not our inference engine. Only default unscaled RoPE and the bias-free SiLU Llama architecture are supported. Cache, sampling and serving are later lessons.

What B8 verifies beyond B7

B7 started every comparison from an HF layer input. B8 instead feeds each of **our layer outputs into our next layer**, then applies our final norm and LM head. Error can accumulate across the stack.

The notebook compares all logits and all row argmax IDs for three prompts. It then checks the two models at every greedy step on the same growing prefix, stopping on EOS or the configured test length. A token mismatch raises even if logits are numerically close.

FP32 comparison uses `rtol=1e-5, atol=1e-4`: accumulated rounding through 22 layers needs a larger absolute allowance than isolated operations. The maximum difference and HF's last-row top-two margin are recorded. Close top candidates can change argmax despite small errors, which is why token equality is a separate required check, not implied by tolerance.

Synthetic tests also cover explicit shifted positions, packed isolation, causality, strict loading, tied weights and greedy parity against HF `generate(use_cache=False)`. A controlled small CUDA FP16 comparison uses the same rounded weights in FP32 with `rtol=atol=1e-2`, not token equality. FP16 logits are returned in FP32, but that cast does not restore lost precision.

Colab memory and scope

Two FP32 TinyLlama copies occupy about 8.8 GB of weight memory plus activations and temporary tensors. Close old GPU-heavy notebook kernels. Use short prompts, and free both models after the run. This notebook is correctness-focused, not a speed comparison or a public generator.

The notebook saves the test summary, environment, prompt comparisons, greedy token IDs and error ranges to `results/B8_full_model.json`. The original payload is archived in B8_full_model.json.

Recorded Colab run (2026-10-31)

All **143 tests passed**, none skipped. All 36 greedy steps (12 per prompt) matched HF exactly. Logits differences across prompts/steps ranged from `7.15e-6` to `1.67e-5`. Packed-vs-separate difference was `4.58e-5`, within tolerance, with equal token choices. Both models had 1,100,048,384 parameters.

An earlier memory-constrained runtime repeatedly lost its kernel. Model loading succeeded on a fresh T4 runtime with 11.10 GiB free system RAM and an almost empty GPU. Resetting a notebook kernel alone is not the same as replacing the whole Colab runtime. Save results before deleting a runtime.

Interview questions

1. **Why is full-model parity necessary?** Correct isolated blocks do not prove correct wiring, weight loading or accumulated stack behavior.
2. **Why require token parity separately from logits tolerance?** Tiny numerical differences can flip the two highest-scoring candidates.
3. **What does HF still do?** Supply trained weights/tokenization and the independent reference. Our forward implements the entire model.

12. Sampling the next token (C1)

Code: sampling.py. Tests: test_c1_sampling.py. Notebook: C1_sampling.ipynb.

Scores are not the choice

The model returns one score per vocabulary token. Sampling decides which token to append. It operates on one `(vocab,)` logits row; C2 will call it on the model's final row.

Toy probabilities: Paris 60%, London 25%, Berlin 10%, Rome 5%. These are synthetic, not measured TinyLlama predictions.

Method	Effect
Greedy	Always choose the largest score
Temperature < 1	Concentrate probability on high scores
Temperature > 1	Flatten probabilities
Top-k	Keep the highest k-score threshold, renormalize
Top-p	Keep the high-probability nucleus reaching the chosen mass, renormalize

With top-k=2 or top-p=0.8 on this example, Paris and London survive: their probabilities become 60/85 and 25/85, about 70.6% and 29.4%. Top-p=0.8 does not mean each retained token gets 80% probability.

Order and conventions

Temperature divides **logits**, not probabilities. Then top-k runs, then top-p runs on the remaining distribution. Softmax produces final weights for `torch.multinomial`.

`sample_token` defaults to greedy (`temperature=0`), which consumes no RNG state. `sampling_probs` defaults to temperature 1 for distribution inspection. `top_k=0` and `top_p=1` disable their filters. Invalid settings still raise even if greedy would otherwise ignore filtering.

Top-k uses the kth score as a cutoff, matching HF. Ties at the cutoff can retain more than k tokens. Greedy ties choose the first (lowest-index) max.

Top-p follows HF's ascending-tail convention: sort ascending, remove tokens whose cumulative probability is <= 1-p, and always keep at least one token. For exact cumulative boundaries this can differ from tutorials that shift a descending `cumsum > p` mask. Tests include the boundary and HF parity.

Precision, randomness and validation

FP16/BF16 scores upcast to FP32 before scaling and softmax. FP64 remains FP64. Negative infinity is accepted as an already-masked score. NaN, positive infinity, all-masked rows and scaling overflow raise clear errors. The input logits are never modified.

A seeded generator must match the logits device. Repeated seeds reproduce results on the same device/software, not necessarily across CPU/GPU or versions. Different seeds change draws, not the probability distribution.

Tests verify hand distributions, temperature, filters/ties, exact HF warper parity, invalid inputs, reproducibility and GPU generator use. An empirical 20,000-draw check uses absolute tolerance 0.015: its largest standard error is about 0.0035, so the bound is over four standard errors. FP32 probability comparisons use absolute 1e-6.

The notebook needs no model weights. It records probability tables and frequency ranges across three independent seeded trials, plus pytest output and environment information, in `results/C1_sampling.json`. The original payload is archived in C1_sampling.json.

Recorded Colab run (2026-10-31)

All **42 tests passed**, none skipped. Same-seed replay matched exactly. Across three 4000-draw trials, observed ranges were Paris 58.98-59.95%, London 24.13-26.18%, Berlin 9.73-10.75%, Rome 5.10-5.18%, consistent with the synthetic target distribution and the statistical tolerance.

Interview questions

1. **Temperature versus top-k/top-p?** Temperature reshapes probabilities; filters remove candidates before sampling.
2. **Top-k versus top-p?** K uses a fixed score-rank cutoff; p adapts the number of candidates to their probability mass.
3. **Does a seed ensure the same output everywhere?** No. It makes a random sequence reproducible within the same device/software conditions.

13. The generation loop (C2)

Code: generation.py. Tests: test_c2_generation.py. Notebook: C2_generation.ipynb.

Grow the context

```
ids = copy(prompt)
repeat up to max_new_tokens:
    positions = 0..len(ids)-1
    logits = model(ids, positions, AttentionMetadata(positions))
    token = sample_token(logits[-1], sampling settings)
    append token to ids
    if token is EOS: stop
```

The model processes the growing prompt on every step. This is deliberately no-cache: C3 will remove recomputation of earlier tokens.

If the prompt is "The capital of France is", the loop might append "Paris", then ".", then EOS. Every choice uses a new model forward with the updated context, not repeated sampling of one unchanged logits row.

Output and stopping contract

`generate` returns a 1-D tensor containing **prompt plus generated tokens**, including a generated EOS. It preserves input dtype/device and does not modify the prompt. Slice `output[prompt.numel():]` for the continuation only.

- `max_new_tokens` counts new tokens, not prompt tokens.
- Zero returns an independent prompt copy without running the model.
- EOS comes from `model.config.eos_token_id`: integer, list/tuple or None.
- A token already in the prompt is not a stopping event. This matters for chat templates with EOS delimiters between prior conversation turns.
- Prompt length plus the entire requested budget must fit the context limit. Oversized requests raise **before forward**, even if generation might have stopped early. This is the user's chosen strict boundary rule.
- Eval mode is required. The loop disables gradients.

The sampler defaults to greedy. Temperature, top-k, top-p and a device-matched generator enable reproducible sampling, with C1's conventions. The shared option validator also runs for zero-token requests.

For CUDA sampling, use `torch.Generator(device=input_ids.device)` to select the prompt's exact GPU. An unindexed `"cuda"` generator is also accepted when the current CUDA device matches the prompt; a different GPU is rejected.

Raw text versus a chat template

Raw text asks the model to continue a document. A chat template formats system/user/assistant turns as expected during chat training. Use `tokenizer.apply_chat_template(..., tokenize=True, add_generation_prompt=True)` to include the correct special tokens once.

With `return_tensors="pt", return_dict=True`, select `["input_ids"][0]` for the single prompt tensor. Indexing the container directly with `[0]` can return a tokenizer `Encoding`, which has no tensor `.to()` method.

A chat template often improves answer style and EOS behavior, but it does not guarantee EOS. The new-token budget is always a separate stopping bound. `skip_special_tokens=True` hides EOS when decoding; the returned IDs still contain it.

Verification

Scripted test models expose each complete prefix, position tensor and chosen token. Tests cover immediate EOS, multiple EOS IDs, prompt EOS, exact budgets, context errors, immutability, seeded sampler agreement and invalid inputs. Small real HF models check token-exact greedy `generate(use_cache=False)` agreement, with both budget stopping and forced first-token EOS.

The notebook runs B8/C1/C2 tests, compares real TinyLlama greedy generation against HF in FP32, then shows a seeded sampled answer using our model. No equality is demanded for stochastic trajectories across differing implementations or devices. Sampler probabilities already have C1 reference checks.

Two FP32 model copies need about 8.8 GB in weight memory. Use a fresh Colab T4 runtime, not several old model kernels. Sampling demos free the HF copy after the greedy comparisons. There are no cache or speed claims yet.

Results, token IDs, stopping observations, test output and environment info are saved in `results/C2_generation.json`.

Colab validation (2026-10-31)

On a Tesla T4 with PyTorch 2.11.0+cu130 and Transformers 5.21.0:

- 96 B8/C1/C2 tests passed; the two-GPU generator rejection test was skipped.
- Both real FP32 greedy sequences matched HF token-for-token.
- Raw continuation used all 12 new tokens and stopped at the budget.
- Chat generation stopped on EOS after 8 new tokens, including EOS.
- Sampled generation with seed 17 replayed exactly and stopped on EOS.
- Zero budget returned the prompt; an oversized context raised as required.

These checks validate the tested no-cache behavior, not cache performance. The next lesson is C3: contiguous KV cache; its implementation is ready and runtime validation is pending.

Interview questions

1. **Why use only the last logits row?** It predicts the token after the entire current prefix; earlier rows predict tokens at earlier positions.
2. **EOS versus max_new_tokens?** EOS is a model-selected end marker; the token budget is an independent hard bound.
3. **Why is this loop wasteful?** It recomputes every earlier token on every step. Causality lets C3 reuse their keys and values instead.

14. Contiguous KV cache (C3)

Code: cache.py, llama.py, generation.py. Tests: test_c3_cache.py. Notebook: C3_kv_cache.ipynb.

Why the old K/V remain valid

Causal attention prevents a token from seeing later tokens. Appending a token does not change an earlier token's embedding, hidden states, queries, keys or values. Reuse the earlier K/V at every layer instead of recomputing them.

This assumes fixed model weights, eval mode, the same request and the same positions. A cache is not valid across weight changes or unrelated prompts. It is inference storage, not an autograd graph.

Shapes and memory

TinyLlama has 22 layers, 32 query heads, 4 KV heads and head dimension 64. Each of our two storage tensors has shape:

```
(22 layers, capacity tokens, 4 KV heads, 64 dimensions)
```

Store four KV heads, not 32 repeated heads. The reference GQA attention still expands the visible prefix temporarily; that is not persistent cache storage.

Bytes per token, across both K and V and every layer:

```
2 * 22 * 4 * 64 * bytes_per_element
FP16: 22,528 bytes = 22 KiB
FP32: 45,056 bytes = 44 KiB
```

A 2,048-token FP16 allocation costs 44 MiB. The equivalent FP32 allocation costs 88 MiB. This counts the two storage tensors, not model weights, attention temporaries or allocator overhead.

Capacity is reserved in advance; `length` is the committed prefix. Do not read `[length:capacity]`: it is uninitialized or belongs to an old request. `reset()` sets length to zero without reallocating or zeroing storage.

Prefill, then decode

For a three-token prompt and four generated tokens:

Forward	Input token count	Absolute positions	Committed length after forward
Prefill	3	0, 1, 2	3
Decode	1	3	4
Decode	1	4	5
Decode	1	5	6

Each forward produces the next token. The final generated token is returned, but does not need another forward unless we want a further prediction. Therefore cache length can be one less than returned sequence length.

Prefill stores every prompt token's K/V. Later forwards project only the new token, attend to all visible cached keys and values, and append its K/V. Queries are not cached. Keys are stored **after RoPE**, once, at their original absolute positions. Values are not rotated.

Do not restart decode positions at zero. Do not rotate old keys again.

Rectangular causal mask

The old no-cache mask was square. With a committed prefix of length 3 and a new two-token chunk at positions 3 and 4:

```
                 key position
               0  1  2  3  4
query at 3     Y  Y  Y  Y  -
query at 4     Y  Y  Y  Y  Y
```

Visibility is `key_position <= query_position`. It is not a fresh `tril(2, 5)`, which would hide most of the old prefix. For one-token decode, the mask has shape `(1, prefix_length + 1)`.

Integration and boundaries

`AttentionMetadata` carries the optional cache without changing the model's three-argument forward. Each decoder layer has a stable storage index; that index adds no parameters and does not change checkpoint keys.

`generate(..., use_cache=True)` creates a fresh cache for each request. The default remains `use_cache=False`. Sampling, output dtype/device, prompt immutability, EOS and the strict context policy remain unchanged. Zero-token requests do not allocate a cache or run forward.

Direct cached forwards require eval mode and `torch.no_grad()`. Positions must append consecutively from `cache.length`; capacity overflow and packed sequence IDs raise before the decoder stack. The cache is bound to its model. Recreate it after moving/casting the model; reset or recreate it after a weight change. Do not share it between concurrent forwards.

Each layer writes an uncommitted suffix. The outer model advances length only after its full forward succeeds. A failed later layer does not alter the committed prefix; a retry overwrites the incomplete suffix.

This is a **single-request contiguous cache**. Packed no-cache forwards still work. Paged storage and multi-request scheduling belong to Part D.

Verification and current status

Tests cover full prefill, single-token and multi-token chunks, MHA/GQA/MQA, cached/no-cache/HF logits, greedy tokens, seeded replay, boundaries, reset, unchanged old entries, hidden unused storage and retry after a layer failure. One CUDA test compares FP16 with the same rounded weights in FP32.

The notebook runs B8/C1/C2/C3 tests on a fresh Colab T4. It compares real FP32 TinyLlama cached logits with both no-cache paths at every greedy step. It separately compares generated sequences with HF's cached generation.

Full-model FP32 comparisons use `rtol=1e-5, atol=1e-4`, as in B8. Changing GEMM row counts can change rounding; token parity is required separately. FP16 uses `rtol=atol=1e-2` on a small controlled model, with no token-exact FP16 claim.

Local checks are static only. Save notebook outputs and the JSON before replacing a Colab runtime. No speed claim is made until C4 measures the no-cache/cached curve.

Recorded Colab run (2026-11-01)

The user-provided C3_kv_cache.json records a Tesla T4 run at commit `7b3969f`, with PyTorch `2.11.0+cu130` and Transformers `5.21.0`.

- **139 B8/C1/C2/C3 tests passed; one test skipped**, because it requires two CUDA devices. The pytest return code is zero and stderr is empty.
- All 20 demonstrated greedy steps matched HF: 12 raw and eight chat. Full generated sequences also matched HF's cached generator.
- The raw prompt prefills six tokens, then forwards one token per step. It generates 12 tokens and ends with cache length 17 versus 18 returned IDs.
- The chat prompt prefills 41 tokens and generates eight, including EOS. It ends with cache length 48 versus 49 returned IDs.

- Cached/no-cache maximum logit differences range from zero to `2.10e-5`. Cached/HF maximum differences range from `7.63e-6` to `3.80e-5`. All recorded argmax checks passed.
- The FP32 demo reserves eight slots: `360,448` bytes total and `45,056` bytes per token (44 KiB). Its committed prefix contains six tokens.
- Reset reuses storage and gives zero logit difference in the demonstrated comparison. Zero budget, strict context errors and position errors pass.
- Seed-17 cached sampling replays exactly.

The JSON was archived from the user's pasted result, preserving its original timestamp and environment. It validates the recorded tests and demos, not unmeasured performance or every possible input.

Interview questions

1. **Why can earlier K/V be reused?** Causality makes earlier hidden states independent of appended tokens, for fixed weights and positions.
2. **Why is the cache 22 KiB per token in FP16?** Both K and V, all 22 layers, four KV heads, 64 dimensions and two bytes per element.
3. **Does caching make decode constant-cost?** No. New-token attention still reads the growing K/V prefix; model weights are still read each step.

15. No-cache versus cached speed curve (C4)

Code: cache_curve.py. Tests: test_c4_bench.py. Plot: plots.py. Notebook: C4_cache_curve.ipynb.

The question

C3 established correctness. C4 asks how much time the cache saves for one request, with different prompt lengths and generation budgets.

Compare our own model with itself. Keep weights, dtype, prompt IDs and new-token count fixed. Change only `use_cache`. This is not an HF comparison, a serving benchmark or an optimized-attention claim.

What caching removes

For prompt length P and N new tokens, no-cache forwards process:

```
P, P+1, P+2, ..., P+N-1
```

Cached forwards process:

```
P, 1, 1, ..., 1
```

Total token rows processed by each decoder layer:

```
no cache: N*P + N*(N-1)/2
cached:   P + N - 1
```

For P=128 and N=32, this is 4,592 versus 159 token rows. That removes repeated old-token projections, MLP work and attention computations. It does **not** predict a 28.9x wall-clock speedup.

Large matrix operations can process many rows efficiently. One-token operations have lower arithmetic intensity and still read model weights. The cached query still reads the growing K/V prefix. Our reference GQA also expands KV heads temporarily and uses Python validation/synchronization. Less arithmetic does not translate directly to the same ratio of elapsed time.

Two controlled sweeps

1. Fix N=32 and vary P across 16, 64, 128, 256 and 512.
2. Fix P=128 and vary N across 1, 8, 16 and 32.

Reuse the shared P=128/N=32 point instead of measuring it twice. Each unique point uses three trials per mode and one full-length warmup per mode. The default experiment has eight unique points.

For N=1, both modes process the prompt once. The cache additionally allocates and writes storage, but has no later forward on which to reuse it. Cache need not win here. With more generated tokens, the prefill cost is spread over more decode steps.

Fixed-length workloads and EOS

A normal chat can stop after eight tokens. Comparing that with a 32-token continuation would confound speed with work performed.

The user approved temporarily disabling EOS stopping for this benchmark. `run_case` first validates the original request/configuration, sets EOS to None, warms up and times both modes, then restores EOS in `finally`. It restores the original value even on failure.

The model can still select the EOS token ID; the benchmark simply continues after it. This is synthetic fixed-length generation, not a chat-quality demo. The normal generation API is unchanged. Do not run concurrent requests on this temporarily modified benchmark model.

FP16 cached/no-cache token choices can differ through rounding. Record the observed equality and token IDs; do not demand token-exact FP16 parity. Both modes must still return exactly P+N IDs. C3's FP32 parity tests remain the correctness evidence.

Timing boundaries

Use `time.perf_counter()` around the public generation call. Synchronize the input's CUDA device immediately before and after the measured call. Without the ending synchronization, queued GPU work can outlive the timer.

The measurement includes request validation, prompt copying, cache allocation, all forwards, greedy sampling, Python overhead and output assembly. It excludes model loading, tokenization, warmup, JSON saving and plotting. It does not measure server queueing, streaming, HTTP or network latency.

Do not synchronize separately after every token; that would change the measured workload. Do not subtract independent prefill/full-run timings to invent a precise decode latency. This lesson reports actual full-call time.

```
generated tokens/s = N / total_seconds
paired speedup    = no_cache_total_ms / cached_total_ms
```

Count new tokens only, not prompt tokens. Above 1 favors cache; below 1 means cache was slower in that paired trial.

Ranges and trial order

Alternate order: no-cache/cached, cached/no-cache, no-cache/cached. This reduces systematic order bias from temperature or clock drift; it does not eliminate noise.

Retain every trial and report min/median/max. Compute speedup within each trial pair, then summarize those ratios. The median paired ratio need not equal the ratio of the two independent medians.

Shaded plot bands show observed min/max, not confidence intervals. No minimum speedup is asserted. A flat curve or regression is a result to investigate, not a reason to clamp values or loosen a test.

Memory

The reserved contiguous K/V storage has capacity P+N:

```
cache storage bytes = (P+N) * 2 * layers * KV_heads * head_dim * dtype_bytes
TinyLlama FP16:       (P+N) * 22 KiB
```

The last generated token is not processed, but capacity still reserves its slot. Report this theoretical storage separately from measured memory.

CUDA allocated peak includes model weights, prompt, outputs and temporary tensors. Incremental peak subtracts allocated memory just before the call. Neither is the same as allocator-reserved memory or total GPU process usage. Cache can add persistent storage while reducing the full-prefix temporaries. Do not interpret incremental peak as cache bytes alone.

Evidence and status

Tests cover exact timing arithmetic with a controlled clock, alternating order, fixed budgets, original EOS restoration on success/failure, invalid requests/timings, a tiny real-model run, CUDA memory accounting and plots. They run in Colab, not locally. Controlled test clocks replace only the benchmark module's local `time` binding, not the shared standard-library clock used by plotting libraries.

The notebook checkpoints each completed point as `status="partial"`, then marks the final JSON complete only after all points and plots succeed. An interrupted experiment is not a complete curve. Save the JSON, PNGs and notebook outputs before replacing the runtime.

Local checks are static only. Next: D1, flattened batching and attention metadata.

Recorded Colab run (2026-11-01)

The user's complete JSON records a Tesla T4 at commit `e50ae86`, with PyTorch `2.11.0+cu130` and Transformers `5.21.0`. **101 C2/C3/C4 tests passed; one two-GPU test skipped.** The earlier plotting-test clock failure was fixed before this run.

All eight unique cases completed, with three paired trials per case. Every measured cached/no-cache generated sequence matched, an observation for these FP16 inputs rather than a universal FP16 parity guarantee.

Prompt sweep, with 32 new tokens:

Prompt tokens	No-cache median ms	Cached median ms	Paired speedup min/median/max
16	1,395.74	1,406.75	0.960 / 0.992 / 1.087
64	1,236.41	1,287.06	0.821 / 0.973 / 0.986
128	1,233.72	1,232.14	0.983 / 0.986 / 1.011
256	1,865.54	1,242.44	1.495 / 1.496 / 1.514
512	3,882.62	1,314.55	2.931 / 2.954 / 2.970

At 512 prompt tokens, generated throughput increases from 8.24 to 24.34 tokens/s. Median incremental allocated peak falls from 198.44 to 130.26 MiB, despite the additional reserved KV storage. This metric includes temporaries, not just KV bytes.

Generation sweep, with 128 prompt tokens:

New tokens	No-cache median ms	Cached median ms	Paired speedup min/median/max
1	38.48	40.82	0.927 / 0.935 / 0.970
8	302.49	316.91	0.939 / 0.955 / 0.978
16	765.86	741.25	0.883 / 1.001 / 1.038
32	1,233.72	1,232.14	0.983 / 0.986 / 1.011

The one-token result shows cache overhead without reuse. At this prompt length, the tested longer generations do not establish a consistent speed advantage. The 16-token case has substantial variation; three trials do not identify its cause.

Short-prompt cache/no-cache times are near break-even, while every paired trial at 256 and 512 prompt tokens favors cache. The first measured prompt length with a clear advantage is 256; the exact crossover is not established. Do not infer that the smaller 64/128-token timings relative to 16 tokens prove those longer prompts are inherently faster.

These curves are consistent with overhead and matrix efficiency masking cache savings for smaller workloads, and avoided recomputation becoming more important for longer prompts. No profiler was captured, so they do not prove a specific bottleneck or universal speedup.

The original JSON payload and metadata were preserved. Raw accounting and summaries were checked statically. The JSON names two generated figures; their PNG contents were not supplied and remain in the user's Colab artifacts.

Interview questions

1. **Why can a cache be slower for one generated token?** There is no later decode forward to benefit; allocation and K/V writes add overhead.
2. **Why does fewer token rows not predict equal wall-clock speedup?** Matrix efficiency, weight/KV traffic, Python overhead and prefill still matter.

3. **What makes this comparison fair?** Same model/dtype/prompt and exact token counts, warmup, CUDA synchronization, alternating order and raw trials.

16. Flattened batching and attention metadata (D1)

Code: batching.py, attention.py, probe.py. Tests: test_d1_batching.py. Notebook: D1_flattened_batching.ipynb.

From one request to several

C3/C4 generate one request at a time. D1 prepares the model to process several independent prompts in one forward, without changing its flattened interface. The model already accepted positions and request IDs in B4/B8; this lesson adds a validated pack/split helper and makes the contract explicit.

This is **no-cache packed prefill**, not continuous batching, batched cached generation or paged storage. The single-request generator remains unchanged.

Three pieces of layout information

For request A with three tokens and request B with two:

```
flat index:    0   1   2   3   4
token IDs:    A0  A1  A2  B0  B1
positions:     0   1   2   0   1
sequence IDs:  0   0   0   1   1
lengths:      (3, 2)
```

- Flat index locates a row in the concatenated tensor.
- Position locates a token within its own request, for RoPE and causality.
- Sequence ID tells attention which request owns that row.
- Lengths route output rows back to requests, in input order.

Sequence IDs are batch-local labels, not token IDs or permanent scheduler handles. Reordering requests can change labels without changing the requests. Positions reset per prompt. Cached decode will later continue each request's own positions, not the batch's flat index.

Why resetting positions is not enough

The existing causal condition is:

```
key_position <= query_position
```

If A0 and B0 both have position zero, this condition lets A0 see B0. The correct packed condition adds request ownership:

```
(key_sequence_id == query_sequence_id)
AND
(key_position <= query_position)
```

For lengths `(3, 2)`, the mask is:

```
              A0 A1 A2 B0 B1
query A0       1  0  0  0  0
query A1       1  1  0  0  0
query A2       1  1  1  0  0
query B0       0  0  0  1  0
query B1       0  0  0  1  1
```

This is block-diagonal causal attention. Each request has its own triangular block, with zero visibility outside it. This mask isolates request data; omitting ownership would allow one prompt to influence another's output.

Packing is not padding

A padded batch often has shape `(requests, longest_length)`. With lengths 3 and 2, it reserves six token slots, including one padding slot.

Our packed IDs have shape `(sum(lengths),)`, here five real tokens. Hidden states have shape `(5, hidden_size)`; logits have shape `(5, vocab_size)`. All tokenwise projections, normalization and MLP work use real token rows.

However, our reference attention still materializes dense scores and masks over the total token count. With N total tokens, it computes N-by-N scores per query head, including cross-request pairs later masked out.

Do not call this a block-sparse kernel. Packing avoids padding rows, but this reference implementation does not skip every masked attention multiplication. No speedup is claimed in D1.

Output routing and next tokens

`pack_prompts(prompts)` returns IDs, `AttentionMetadata` and a lengths tuple. `split_packed(logits, lengths)` returns token-major views in request order. The returned views share the packed tensor's storage; do not mutate them if the packed result must remain unchanged.

For lengths `(3, 2)`:

```
request A logits = packed_logits[0:3]
request B logits = packed_logits[3:5]
A next token     = argmax(packed_logits[2])
B next token     = argmax(packed_logits[4])
```

Using only `packed_logits[-1]` would predict for B and omit A. Each request's last row predicts its own next token. A single packed forward predicts one next token per request; it is not a complete multi-request generation loop.

Why the model need not be rewritten

Embedding, RMSNorm, projections and SwiGLU already operate on flattened token rows. They do not mix independent requests. Attention is the operation that mixes tokens; positions and sequence IDs control that mixing at every layer.

Correct packed outputs should match separate requests, within floating-point tolerance. Changing one request's token content while retaining the same layout must not change any other request's outputs. Permuting requests should permute the routed outputs, with possible small changes from GEMM rounding.

All requests share this model's weights, device and dtype. The helper requires nonempty integer prompts on one device with one integer dtype. It preserves input tensors; the model checks vocabulary bounds.

The context limit applies to each request's length/positions, not the sum of unrelated prompt lengths. Large total batches can still exceed available memory because the reference attention is dense.

Boundaries with C3 and D2

C3's contiguous cache tracks one committed length for one request. It cannot represent several requests advancing independently; packed sequence IDs still cause that cached path to raise explicitly.

D2 introduces per-request block tables and slot mappings. Later scheduling decides which requests contribute prefill/decode tokens to a batch. D1 only establishes the no-cache ownership, position and output-routing contract.

Verification and status

Tests cover exact layout/mask values, round trips, dtype/device checks, output views, last-row routing, invalid inputs, MHA/GQA/MQA, separate/HF full-model parity, cross-request isolation, within-request causality, permutation, individual context limits and continued rejection of packed contiguous cache.

The notebook demonstrates deliberately missing request IDs with tiny hand values, then compares real FP32 TinyLlama packed logits with separate ours/HF for every row. Full-stack assertions use B8's `rtol=1e-5, atol=1e-4`. Argmax equality is required separately. A small CUDA FP16 test compares the same rounded weights in FP32 with `rtol=atol=1e-2`, without token-exact claims.

Local checks are static only. Save `D1_flattened_batching.json` and the notebook with outputs. This lesson makes no batching performance claim.

Recorded Colab run (2026-11-01)

The user's D1_flattened_batching.json records a Tesla T4 at source commit `0044311`, with PyTorch `2.11.0+cu130` and Transformers `5.21.0`. All **140 B4/B8/C3/D1 tests passed**, none skipped. The pytest return code is zero and stderr is empty.

- Request lengths `[6, 6, 9]` produce 21 packed token rows versus 27 padded slots: six padding slots avoided, not a measured performance improvement.
- Packed/separate and packed/HF maximum logit differences range from `1.81e-5` to `2.69e-5`, within the full-stack tolerance.
- Every recorded logits-row argmax agrees. The routed next tokens are `Paris`, `Berlin` and `providing`, one for each respective prompt.
- Changing the second prompt produces zero differences on both untouched requests. Reordering requests as `[2, 0, 1]` also produces zero differences after routing back to the originals in this run.
- The synthetic ownership demo gives `[1, 1, 10]` with correct request IDs, versus `[5.5, 4, 5.5]` without them. Reset positions alone are insufficient.
- Packed C3 cache use raises `contiguous cache supports one request, not packed sequence_ids`, as required.

The original pasted JSON was archived without regenerating results or changing notebook outputs. These checks validate the demonstrated no-cache packing contract, not a scheduler, paged cache or arbitrary batch performance. Next: D2, block tables, slot mapping and paged cache.

Interview questions

1. **Why are positions and sequence IDs separate?** Positions control RoPE and causal order; IDs prevent attention between independent requests.
2. **Why not select only the final packed logits row?** That row belongs only to the last request. Each request needs its own last row.
3. **Does a dense block-diagonal mask skip cross-request computation?** No. It removes visibility, but our dense reference still computes masked scores.

17. Block tables, slot mapping and paged KV cache (D2)

Code: paged_cache.py, attention.py, llama.py, probe.py. Tests: test_d2_paged_cache.py. Notebook: D2_paged_cache.ipynb.

Why change the storage layout?

C3 reserved one contiguous token array per request. D1 showed how to mix independent token rows in a forward. D2 combines independent cached requests with a shared pool of fixed-size physical KV blocks.

A request's context remains in logical token order, even if its physical blocks are scattered. Attention still sees the same keys, values and causal positions. Paging changes addressing, not the attention equation.

The user approved manually assigned, disjoint blocks for D2. This lesson has no free list, allocation policy, reference counts, shared-prefix blocks, copy-on-write, scheduler or performance kernel. D3 will manage allocation.

Logical position versus physical location

For block size B and token position p:

```
logical block = p // B
offset        = p % B
physical block = request_block_table[logical block]
physical slot  = physical block * B + offset
```

With B=2 and request 7's block table `[4, 1, 5]`:

Logical position	Logical block	Offset	Physical block	Physical slot
0	0	0	4	8
1	0	1	4	9
2	1	0	1	2
3	1	1	1	3
4	2	0	5	10
5	2	1	5	11

Sorting the physical blocks would corrupt the logical order. Advancing to position 2 crosses from physical block 4 to block 1, not necessarily the next physical block. A scrambled-block test catches accidental contiguous addressing assumptions.

RoPE uses logical position p, never the physical slot. Moving a block changes its address, not a token's meaning or rotation. Keys remain stored after RoPE; values are not rotated.

Shared pool and per-request lengths

Each storage tensor has shape:

```
(layers, physical_blocks, block_size, KV_heads, head_dim)
```

It still stores KV heads, not expanded query heads. Total K/V allocation is:

```
2 * layers * physical_blocks * block_size * KV_heads * head_dim * dtype_bytes
```

TinyLlama costs 22 KiB per physical token slot in FP16 or 44 KiB in FP32. The whole pool is preallocated in D2, including unassigned blocks. Assignment does not allocate a new tensor. This lesson does not measure a memory saving.

Every request has an independent committed length. A token batch might contain:

```
request IDs:   [7, 9, 9, 9]
positions:     [6, 0, 1, 2]
```

Request 7 decodes one token after its six-token prefix; request 9 prefills three tokens. Another request can be paused and omitted from the batch. The flat row index is neither logical position nor physical address.

Readable paged attention

For every layer:

1. Project new Q/K/V and apply RoPE with logical positions.
2. Use slot mapping to write new K/V into the shared pool.
3. Group new queries by request ownership.
4. Gather that request's visible logical prefix through its block table.
5. Build `key_position <= query_position` within that request.
6. Run the existing reference GQA attention.
7. Scatter outputs back into the original flat query-row order.

For a two-token chunk after length 3:

```
query position 3 sees keys 0,1,2,3
query position 4 sees keys 0,1,2,3,4
```

The first new query must not see the later token in its own chunk. No query sees another request's blocks. Unwritten tail slots and unassigned blocks never participate in the gathered prefix.

This PyTorch path gathers temporary contiguous K/V copies and expands KV heads for reference GQA. It is not a fast paged-attention kernel. Mixed prefill/decode works correctly here, but D2 makes no throughput claim.

Commit, retry and reset

New positions must be consecutive from each active request's committed length, in that request's appearance order. Interleaving request rows is allowed; skipping, duplicating or restarting committed positions is not.

All layers write uncommitted append slots. The outer model advances all active lengths only after the entire forward and logits succeed. A failure in a later layer leaves lengths and committed-prefix bytes unchanged. Retry overwrites the incomplete append slots.

`reset(request_id)` sets just that request's length to zero, without zeroing or freeing its blocks. Its old contents are hidden and overwritten on reuse. Other request lengths remain unchanged. Freeing blocks belongs to D3.

Do not share a cache between concurrent forwards, change weights underneath it, or alter a committed prompt. Reset/recreate after weight changes; recreate after moving or casting the model. The cache is bound to one model, dtype and device and requires eval mode with gradients disabled.

Validation and interface

`new_paged_cache(num_blocks, block_size, block_tables)` constructs the pool. `AttentionMetadata(..., paged_cache=pool)` selects the gather path, using explicit request IDs and positions. The three-argument model interface and checkpoint parameter names remain unchanged.

Contiguous and paged caches are mutually exclusive. Existing no-cache packed and C3 contiguous paths stay supported. The public single-request generator still offers contiguous caching only; D2 demos use explicit cached forwards.

Tables are copied to immutable tuples and exposed read-only. Invalid block IDs, repeated/shared blocks, unknown request IDs, missing layout tensors, capacity overflow, invalid append positions and context overflow raise. Per-request table capacity and model context length are separate limits.

No shared blocks are permitted, even for common prefixes. Without reference counts and copy-on-write, allowing them could corrupt independent requests.

Verification and status

Tests cover hand slot values, scrambled block sizes 1-4, MHA/GQA/MQA, contiguous/HF parity, multi-token boundary crossing, mixed new-prefill/decode, paused requests, interleaved output routing, reset, poisoned unused/other blocks, failure/retry and explicit boundary errors.

The notebook compares ordered and scrambled pools against separate no-cache ours/HF on real FP32 TinyLlama. Full-stack assertions use `rtol=1e-5, atol=1e-4`, with independent all-row argmax checks. A small CUDA FP16 test uses the same rounded weights in FP32 with `rtol=atol=1e-2`; it does not require token-exact FP16 parity.

Local checks are static only. Save `D2_paged_cache.json` and notebook outputs. Next: D3, block manager free list and reference counts.

Recorded Colab run (2026-11-03)

The user's D2_paged_cache.json records a Tesla T4 at commit `5f14f9c`, with PyTorch `2.11.0+cu130` and Transformers `5.22.0`. **197 B8/C2/C3/D1/D2 tests passed; one C2 test requiring two CUDA devices was skipped.** Pytest returned zero with empty stderr.

Both ordered and scrambled pools passed the following stages:

Stage	Active work	Committed lengths afterward: 0, 1, 2
Initial prefill	Six prompt tokens each for requests 0 and 1	6, 6, 0
Mixed	One decode token for 0, nine prompt tokens for 2; pause 1	7, 6, 9
Reordered decode	One token each, in request order 2, 1, 0	8, 7, 10

Across 14 request comparisons (seven per physical layout), all tested logits rows matched the no-cache/HF argmax choices. Paged/no-cache maximum differences range from zero to `4.58e-5`; paged/HF differences range from `8.58e-6` to `4.58e-5`, within the stated FP32 tolerance.

The seven-block, four-slot FP32 pool costs `1,261,568` bytes per layout. The hand mapping gives physical slots `[8, 9, 2, 3, 10, 11, 4, 5, 0, 1]` for the demonstrated two-block-table layout.

Overlapping block ownership and restarting a committed request's positions raise the expected errors. Reset reuses pool storage and gives zero logits difference against no-cache in the demonstrated comparison.

The original pasted JSON and environment metadata were preserved. Static record checks verified slots, lengths, ranges and byte accounting; no local runtime tests were run. These observations validate the tested gather path, not allocation, shared blocks, scheduling or a performance kernel.

Interview questions

1. **What does a block table translate?** A request's logical block number into a physical block ID; the within-block offset is unchanged.
2. **Why scramble the physical blocks in a test?** Correctness must not depend on physically contiguous or sorted storage.
3. **Why does D2 reject shared blocks?** Independent writes require ownership management and copy-on-write rules, which this lesson does not implement.

18. Block ownership, reference counts and reuse (D3)

Code: block_manager.py, paged_cache.py, llama.py. Tests: test_d3_block_manager.py. Notebook: D3_block_manager.ipynb.

What changes after D2?

D2 translated logical positions through manually assigned, disjoint block tables. D3 assigns those tables from a free list, grows them when a request needs more capacity, and returns blocks when that request finishes. Attention, logical RoPE positions and successful-forward length commits remain the D2 implementation.

This is a single-threaded ownership layer, not a scheduler. It does not choose which request runs, evict live requests, preempt, share model prefixes or implement copy-on-write. The public generator still uses the C3 contiguous cache. D2's manual `new_paged_cache(...)` interface remains supported.

The allocator contract

`BlockAllocator(num_blocks)` starts with every physical block free. `allocate(count)` returns distinct block IDs, each with reference count one. It either satisfies the whole reservation or raises `BlockPoolExhausted` without changing state. Zero/negative/noninteger sizes raise `ValueError`.

`retain(block)` adds a reference to a live block. `release(block)` removes one; only the transition from one reference to zero returns it to the free list. Invalid IDs, retaining a free block and releasing a free block raise.

For a three-block pool:

```
operation              references     next-allocation order
start                  [0, 0, 0]      [0, 1, 2]
allocate(2)            [1, 1, 0]      [2]
retain(0)              [2, 1, 0]      [2]
release(0)             [1, 1, 0]      [2]
release(0)             [0, 1, 0]      [0, 2]
allocate(1) -> (0,)    [1, 1, 0]      [2]
```

The free list is a stack: allocation initially visits ascending IDs, then prefers recently released blocks. Correctness does not depend on that order. The sum of references need not equal the number of live blocks.

At every operation:

```
free blocks = exactly the IDs whose reference count is zero
number of free blocks + number of live physical blocks = pool size
```

The allocator tests exercise counts above one. Managed model requests remain disjoint, with exactly one reference per assigned block. Reference counts alone do not make shared-prefix writes safe: an independently writable shared tail would still need copy-on-write and compatible prefix metadata.

Reservation is not committed length

Create a fixed tensor pool with `model.new_managed_paged_cache(num_blocks, block_size)`. It starts without requests. Before forwarding, call `pool.reserve(request_id, total_tokens)`. The argument is the total processed-token capacity, not the append count.

For B=2, a request committed at length 3 already owns two blocks. Reserving capacity 5 adds one block:

```
before: table [0, 1],    committed length 3, capacity 4
reserve(0, 5)
after:  table [0, 1, 3], committed length 3, capacity 6
successful append of positions [3, 4]
after:  table [0, 1, 3], committed length 5, capacity 6
```

Required blocks are `ceil(total_tokens / B)`. Existing blocks are retained; reservation does not shrink a table. A reservation below the committed length or above the model context limit raises. Tables and lengths are read-only views; block lists are immutable tuples.

Admission and growth allocate all missing blocks before updating a table. On exhaustion, no partial table, length or reference-count change is published. Batch callers reserve requests explicitly; reservations are atomic per request, not a transaction across a whole batch. Forward never allocates implicitly.

Failure, reset and release

If a later decoder layer fails, append slots may contain incomplete K/V, but the committed lengths and prefix K/V remain unchanged. Reserved blocks stay owned for retry. Retry writes the same uncommitted positions again.

`reset(request_id)` hides the old prefix by setting its length to zero and keeps all its blocks. `release_request(request_id)` removes its table and length, then returns its blocks to the allocator. Unknown request IDs raise.

Release does not zero K/V bytes or shrink GPU tensor storage. A new request can reuse those physical IDs because attention gathers only its written logical prefix. The notebook and tests poison unused storage with NaN and compare reused requests with fresh no-cache/HF forwards.

Do not use one pool concurrently, change weights under committed prefixes, or externally mutate its tensors. Recreate after moving/casting the model. This educational cache is not a secure memory-erasure mechanism.

Memory and performance boundaries

Both K and V retain shape:

```
(layers, num_blocks, block_size, KV_heads, head_dim)
```

The whole pool is allocated once. Reserving and releasing blocks change ownership, not GPU allocated bytes. TinyLlama uses 44 KiB per FP32 physical token slot (22 KiB in FP16). D3 measures no memory saving or throughput gain. The reference attention still gathers temporary K/V copies.

Validation

Randomized property tests compare allocator reference counts with an independent counter model across 20 seeds and 400 operations each. Ten additional seeded 200-operation traces check disjoint managed tables, growth and conservation. Boundary tests check invalid input, exhaustion without partial allocation, reset versus release, read-only metadata and fixed storage addresses.

Small CPU/CUDA model tests check mixed growth/reuse against separate FP32 ours/HF references, plus injected later-layer failure and retry. The Colab notebook repeats lifecycle checks with real FP32 TinyLlama and saves `D3_block_manager.json` through the existing environment recorder. All-row logits use D2's `rtol=1e-5, atol=1e-4` tolerance, with separate exact argmax checks; this is numerical correctness, not a timing benchmark.

Local validation is static only. Save both the JSON and successful notebook outputs before replacing a Colab runtime.

Recorded Colab run (2026-11-04)

The user's complete D3_block_manager.json records a Tesla T4 at commit `e176350`, Python `3.13.16`, PyTorch `2.11.0+cu130` and Transformers `5.22.0`. **253 B8/C2/C3/D1/D2/D3 tests passed; one C2 test requiring two GPUs was skipped.** Pytest returned zero with empty stderr. These are user-supplied runtime results, not local tests.

Both failed reservations required two additional blocks when only one was free, and left state unchanged. Growing request 0 then assigned its third block without advancing its length. The injected later-layer failure kept lengths at `{0: 3, 1: 2}` and retained the same tables and reference counts. The result reports preserved committed-prefix K/V, and the explicit retry comparison records successful processing of positions 3 and 4.

Successful comparison stage	Request 0 length	Request 1 length	Request 2 length
Initial prefill	3	2	-
Retry boundary-crossing append	5	2	-
Mixed new prefill and resumed append	Released	3	4
Reset without release, then process two tokens	Released	3	2

Across six request comparisons and fourteen new logits rows, all tested argmax choices matched separate no-cache ours/HF. The paged/no-cache maximum absolute differences range from zero to `5.72e-6`; paged/HF differences range from `3.81e-6` to `6.68e-6`.

Request 2 reused physical blocks `[3, 1]`. The four-block, two-slot FP32 pool occupied `360,448` bytes (352 KiB), with unchanged K/V storage addresses. The final snapshot contains no requests, four free blocks and zero references. This validates the demonstrated ownership lifecycle; it is not a throughput or GPU allocation reduction measurement. D4's separate validated run is archived in D4_scheduler.json.

Three interview questions

1. **Why reference counts, not just a free flag?** A block remains live until its final owner releases it; multiple leases cannot free it prematurely.
2. **Why separate reservation from committed length?** Capacity can grow before execution, while only a complete successful forward makes tokens visible. A failed forward can retry without exposing incomplete K/V.
3. **Does releasing a request free GPU memory?** It returns physical blocks for reuse within the fixed pool; it does not deallocate the tensor.

Next: D4, continuous batching and preemption policy.

19. Continuous batching and recomputation preemption (D4)

Code: scheduler.py, block_manager.py, paged_cache.py. Tests: test_d4_scheduler.py. Notebook: D4_scheduler.ipynb.

What is the scheduler deciding?

D3 answers, "Which physical blocks belong to this request?" D4 answers, "Which requests should process tokens in the next forward?"

A scheduler step builds a **batch plan**. Each item identifies one request, the logical positions to process, and whether that work is prefill, decode or recomputation. The scheduler reserves the necessary cache capacity. It does not run the model, select tokens or calculate logits.

The notebook executes each plan explicitly through the existing D2 parity helper, then acknowledges the successful step. D5 will build the automatic engine loop and size its memory pool. The public single-request generator does not change in D4.

Why continuous batching?

With static batching, a fixed group starts together. A short request may finish while a long request still runs. If new arrivals must wait for the whole group to finish, available batch capacity goes unused.

Continuous batching revisits membership between forwards. Finished requests release their blocks. New requests can join the next plan while older ones continue. A batch can mix one request's prompt processing with another request's one-token append.

This implementation has a deliberately small policy:

- Arrival order sets priority: first come, first served (FCFS).
- `max_batch_tokens` limits the sum of new token rows in a forward, not the number of requests or their full cached lengths.
- Prefill consumes up to the remaining token budget. Large prompts can therefore span several forwards.
- Decode processes at most one new token per request per step.
- Each request appears at most once in a plan. Rebuilding a prefix and decoding its next token happen in separate steps.

Oldest-first priority is simple, not a latency or fairness optimum. A large older prefill can consume the whole budget while newer work waits.

Three meanings of "length"

Suppose a request processed two tokens, then lost its blocks to preemption:

Quantity	Before preemption	After preemption
Logical progress: tokens already processed successfully	2	2
Resident cache length: valid token entries currently in K/V	2	No cache; rebuilding starts at 0
Reserved physical capacity	2 or more slots	0 slots

`RequestState.processed_tokens` preserves the logical progress. `cache.lengths` describes only resident K/V. Neither is a count of references.

The caller retains token IDs outside the KV cache. Replaying those IDs reconstructs the prefix without choosing new tokens or adding them to the logical history again. D4's demos retain fixed tokenized prompt slices.

First worked example: a new arrival joins an older request

Use five blocks with two slots each and a four-token batch budget:

- Request 0 has a three-token prompt and two one-token append steps.
- Request 1 has a two-token prompt and no append steps.
- Request 2 arrives after the first forward, with a three-token prompt.

Ranges below are logical positions; the end is exclusive.

Step	Planned work	Token rows	Finished after success
0	0: prefill `[0,3)`; 1: prefill `[0,1)`	4	Nobody
1	0: decode `[3,4)`; 1: prefill `[1,2)`; 2: prefill `[0,2)`	4	Request 1
2	0: decode `[4,5)`; 2: prefill `[2,3)`	2	Requests 0 and 2

Request 2 starts in step 1, before request 0 finishes. That changing membership is continuous batching. Request 1's prompt is chunked because step 0 has only one token of budget left.

What if there are not enough blocks?

First, reject any request whose complete finite workload cannot fit in the entire pool by itself, or exceeds the model context limit. This prevents a request from waiting forever for impossible capacity.

For feasible requests, an older request may still need a block held by a newer resident request. The scheduler can **preempt** the newer request:

1. Keep its logical progress and the caller's token history.
2. Release all of its resident blocks through the D3 cache.
3. Give the needed blocks to higher-priority work.
4. Later reserve space again and replay the missing prefix.

This is release-and-recompute, not a GPU-to-CPU swap. Recomputing costs model work. Copying/swapping would need a different memory and transfer policy.

Victims are considered newest first, and only among requests newer than the current candidate. Already selected work is never evicted. The scheduler checks that enough space can be reclaimed before releasing any victim. If an unselected lower-priority request cannot fit, it waits without evicting anyone merely for an unsuccessful reservation.

Second worked example: an actual preemption

Use only three blocks with two slots each and a three-token batch budget. Requests 0 and 1 each have a two-token prompt and three append steps. Each can fit alone: five tokens require three blocks. Together they eventually need more blocks than the pool has.

Step	Planned work	Explanation
0	0: prefill two tokens; 1: prefill one	Budget is exhausted
1	0: decode position 2; 1: finish prefill	All three blocks are assigned
2	0: decode position 3	1 cannot get its second block yet
3	Preempt 1; 0: decode position 4	0 needs its third block to finish
4	1: recompute positions 0 and 1	0 has released its blocks
5	1: decode position 2	Rebuilt prefix is ready
6	1: decode position 3	Continue normally
7	1: decode position 4	1 finishes and releases its blocks

At step 3, request 1 still has logical progress two, but no resident cache. At step 4, the same first two tokens are processed again. Their replay does not advance logical progress past two. Step 5 does new work and advances it to three.

The workload contains ten distinct logical token positions across both requests. The model processes twelve rows: ten original rows plus two recomputed rows. Preemption enables progress in a small pool, but is not free.

Execution, failure and cancellation

The caller uses this sequence:

```
plan = scheduler.schedule()
# Execute every plan.work item in one successful paged forward.
scheduler.complete_step()
```

`schedule()` reserves space but does not commit token lengths. `complete_step()` checks the entire batch's expected cache lengths before updating any request progress. Finished requests release their blocks only after that check succeeds.

If forward fails, do not acknowledge it. The model leaves resident committed lengths unchanged, and the same pending plan is returned for retry. Reservations and any preemptions already performed remain in effect. Retrying overwrites incomplete append slots. This is not a transaction that rolls scheduling back.

Arrivals and cancellations are accepted only between completed steps. Changing request membership while a plan is pending raises. Cancelling waiting, running or preempted work makes it terminal and releases any resident blocks. IDs cannot be reused during one scheduler instance. Request snapshots and plans are immutable.

The scheduler owns an initially empty managed cache. Do not separately reserve, reset, release or forward requests behind its back. One scheduler is single-threaded and permits one pending batch at a time.

Finite teaching workloads, not a serving API

`add_request(request_id, prompt_tokens, decode_steps=0)` registers work counts. `decode_steps` means additional token positions to process after the prompt; it is not a `max_new_tokens` sampling budget. Sampling one output from the prompt does not itself append that output to the KV cache.

The caller supplies and retains actual token IDs, including any previously chosen output tokens needed for recomputation. The D4 notebook supplies fixed suffixes to keep policy and cache correctness separate from generation. Token selection, EOS handling, streaming, a production request lifecycle and automatic GPU memory sizing remain outside this lesson.

Validation and status

Tests cover exact worked timelines, chunking, mixed batches, dynamic arrivals, newest-first eviction, deferred requests, cancellation, input validation, immutable metadata and failed-step acknowledgement. Twenty randomized traces check bounded plans, exclusive ownership, monotonically increasing logical progress and termination for finite admitted workloads.

Small CPU/CUDA model tests execute the preemption scenario against no-cache ours/HF references, inject a later-layer failure, and retry. The notebook repeats both worked timelines with real FP32 TinyLlama. Every new or replayed logits row uses `rtol=1e-5, atol=1e-4` plus exact argmax checks.

Local checks are static only. D3's complete real-model failure/retry evidence is archived separately in D3_block_manager.json. The notebook saves `D4_scheduler.json` with environment details, test output, both complete timelines, preemption/recomputation records and numerical ranges. It checks

required scenario records before saving rather than relying only on success flags. Preserve the successful notebook with outputs as well.

No throughput improvement, latency bound or GPU allocation reduction is claimed. The cache pool remains fixed and attention still uses readable PyTorch gathers.

Recorded Colab run (2026-11-04)
The user's D4_scheduler.json records timestamp `2026-11-04T12:54:48Z`, commit `47ea839`, a Tesla T4, Python `3.13.16`, PyTorch `2.11.0+cu130` and Transformers `5.22.0`. **304 B8/C2/C3/D1/D2/D3/D4 tests passed; one C2 test requiring two GPUs was skipped.** Pytest returned zero with empty stderr. Both CPU and CUDA scheduled-model preemption/failure/retry tests passed in this run. This is user-supplied runtime evidence, not a local test execution.

Both real-model timelines match the worked examples above:

Scenario	Blocks, B=2	Batch budget	Steps	Logical token positions	Model rows	Replay rows
Continuous batching	5	4	3	10	10	0
Preemption/ recomputation	3	3	8	10	12	2

In the first scenario, request 2 joins step 1 while request 0 is still running. In the second, step 3 preempts request 1 with logical progress two and no resident cache. Step 4 rebuilds positions 0 and 1: resident length returns to two, while logical progress remains two. Step 5 then decodes position 2. Request 1 has exactly one recorded preemption. All requests finish and both final snapshots contain empty block tables and fully free pools.

Seventeen request comparisons cover 22 new/replayed logits rows. All recorded argmax choices match no-cache ours/HF references. Paged/no-cache maximum absolute differences range from zero to `2.50e-5`; paged/HF differences range from `3.81e-6` to `2.50e-5`, within the stated FP32 tolerances.

The five-block pool occupies `450,560` bytes (440 KiB); the three-block pool occupies `270,336` bytes (264 KiB). These are fixed allocations for different scenarios, not a measured reduction from releasing a request. Extra replay rows demonstrate a recomputation cost, not a timing or throughput result. Preserve the successful notebook outputs separately from the archived JSON.

Three interview questions

1. **What makes batching continuous?** Batch membership can change between forwards; new arrivals need not wait for an entire old group to finish.
2. **What survives recomputation preemption?** Logical progress and retained token IDs survive; resident K/V blocks are released and later rebuilt.
3. **Why might a smaller pool increase work?** Eviction can force prefix recomputation, adding token rows that an ample pool would not need.

Next: D5, the engine loop and memory budgeting.

20. Engine loop and memory budgeting (D5)

Code: engine.py, memory.py, scheduler.py. Tests: test_d5_engine.py. Notebook: D5_engine.ipynb.

From a scheduling decision to generated text

D4 makes a plan and reserves blocks. Its caller still has to supply token IDs, execute the model and acknowledge success. D5 supplies that caller: the **inference engine**. A request contains a prompt, sampling options, token history and a finite output budget.

The engine owns one evaluated model, one managed paged KV pool and one FCFS scheduler. It uses the same flattened model interface:

```
input_ids: (N,)
positions: (N,)
sequence_ids: (N,)
logits: (N, vocabulary_size)
```

For each plan, it concatenates the scheduled slices from each request. Positions start at that request's resident cache length. Sequence IDs identify ownership; they are not token IDs or positions. The existing paged attention backend prevents attention between unrelated requests.

The operation order is:

```
schedule and reserve
    -> pack token IDs, positions and owners
    -> model forward and whole-batch KV commit
    -> route final logits rows and stage sampled tokens
    -> scheduler.complete_step()
    -> publish histories and release early-EOS requests
```

There is no model rewrite, HF forward inside our engine, server or background worker. The C2/C3 public `generate()` paths remain supported. Admission validation is shared with them.

Processed tokens are not sampled outputs

Let the prompt length be `P` and the requested output length be `G > 0`. The engine schedules `P + G - 1` model rows before any recomputation. The final output has `P + G` token IDs.

For `P=3`, `G=3` and a large enough batch budget:

Forward	Input token positions	Sampled output	Resident entries needed
Prefill	0, 1, 2	First new token, position 3	3
Decode	3	Second new token, position 4	4
Decode	4	Third new token, position 5	5

The final sampled token is returned without another forward. It does not need a KV entry. Therefore D4 receives `decode_steps=G-1`, not `G`. The engine deliberately preserves C2's stricter admission rule: `P + G <= max_position_embeddings`, even though the last output is not processed. Oversized requests raise instead of silently shortening output.

Chunked prefill only samples when the chunk reaches the end of the prompt. Recomputation never samples. Sampling a partial prompt or a rebuilt prefix would produce extra outputs and consume extra random draws.

For `G=0`, the engine returns an independent prompt snapshot immediately. It creates no scheduler request and reserves no blocks for that request. Engine construction still allocates its shared pool.

Continuous arrivals and cancellation

The minimal API is:

```
from tinyserve.engine.engine import InferenceEngine

engine = InferenceEngine(model, kv_budget_bytes=8 * 1024**2,
                         block_size=16, max_batch_tokens=32)
engine.add_request(0, prompt_ids, max_new_tokens=8)
updates = engine.step()
engine.add_request(1, another_prompt_ids, max_new_tokens=4)
results = engine.run()
```

The byte budget is KV-only, not a total process-memory limit. The example is not a universal safe configuration; a request must fit in the chosen pool by itself.

`step()` returns immutable, cumulative `RequestOutput` snapshots for requests that sampled during that step. A partial prompt or replay step can return an empty tuple while work remains. Use `has_pending_requests`, not whether the latest update tuple is empty, to decide when work is finished.

Each output contains the request ID, full token history, generated tokens and `finish_reason`: `None`, `"length"`, `"eos"` or `"cancelled"`. `get_output()` returns a snapshot at any time. `run()` drains finite queued work and returns all retained results in arrival order. Use `step()` when new requests must join between forwards.

EOS is included in the output. Prompt EOS does not stop a new turn. An early EOS cancels the remaining scheduled count after the successful batch is acknowledged, returning its blocks. EOS on the final scheduled forward reports `"eos"` while D4 already regards the work as finished.

Cancellation accepts waiting, running or preempted requests between completed steps. It retains tokens already emitted and releases resident blocks. Terminal IDs cannot be reused, including zero-output requests. IDs must fit the int64 sequence-ID representation.

The model must stay in eval mode on its original device and dtype. The engine and scheduler exclusively own their cache. Do not reserve, reset, release or commit its requests externally.

Preemption keeps history, not random decisions

Use two requests with `P=2`, `G=4`, a three-block pool, two slots per block and a three-row batch budget. Each needs five processed positions, so either fits alone. Their combined block demand eventually exceeds the pool.

The scheduler releases the newer request's K/V. Its history still contains the prompt and the output already selected from that prompt. Later it replays the two processed prompt tokens. The engine emits nothing during replay. Its next decode processes the previously sampled output token, then selects the next output.

As in D4's worked timeline, ten distinct model positions become twelve executed rows because two rows are replayed. The engine still returns four outputs per request, not five. Per-request `torch.Generator` instances keep seeded sampling independent of interleaving. A shared generator or the default global RNG intentionally depends on scheduling order. Reproducibility is limited to the same device and software environment.

Failure is not success

A later-layer forward failure leaves committed cache lengths unchanged. Reservations and preemptions remain in effect. The same plan is pending for a retry, and histories and RNG states have not advanced. Arrivals and cancellation are blocked until that pending step succeeds.

After a successful model forward, K/V has already committed. The engine therefore retains logits until sampling and acknowledgement succeed. If sampling fails, it restores generator states, retains unchanged token histories and propagates the error. Retrying sampling uses those saved scores, not another forward against an already appended cache.

Invalid saved logits are not recoverable by blindly retrying. Discard the engine and diagnose the model or numerical settings. The retry contract requires the model's D2/D3 whole-forward commit behavior; arbitrary hooks that raise after cache commit do not qualify as pre-commit failures.

This is single-threaded inference, not request isolation for an exposed service. Timeouts, transport errors and asynchronous cancellation belong to later serving work.

KV memory: count KV heads, not query heads

One cached token stores keys and values at every layer:

```
bytes_per_token = 2 * layers * KV_heads * head_dim * element_bytes
bytes_per_block = block_size * bytes_per_token
num_blocks      = kv_budget_bytes // bytes_per_block
allocated_bytes = num_blocks * bytes_per_block
unused_bytes    = kv_budget_bytes - allocated_bytes
```

TinyLlama has 22 layers, four KV heads and head dimension 64:

Model dtype	Bytes per token	KiB per token	Bytes per 16-slot block
FP32	45,056	44	720,896
FP16	22,528	22	360,448

The 32 query heads do not multiply KV storage. GQA is why only four heads are cached. The planner derives element size from the model's embedding weights, matching the existing cache allocation mechanism.

An FP16 KV-only budget of 1 MiB fits two 16-slot blocks: `1,048,576 // 360,448 = 2`. The allocation is 720,896 bytes and 327,680 budget bytes remain unused. Three blocks would exceed the budget. The remainder is budget rounding, not unused slots inside a live block. Within each request's last block, up to `block_size-1` slots may be unused.

`plan_kv_memory()` does not allocate tensors. It rejects budgets that cannot fit even one block. The engine verifies that the resulting pool's actual tensor bytes equal the plan. Releasing requests returns blocks for reuse; it does not shrink the underlying K/V tensors or return their allocation to the CUDA driver.

Budgeting against a live GPU

Load model weights first. Then:

```
engine = InferenceEngine.from_available_memory(
    model, reserve_bytes=1024**3, block_size=16, max_batch_tokens=32,
)
```

The class method queries CUDA driver free memory on the model's device and subtracts the explicit reserve before rounding to blocks. CPU models must use an explicit KV byte budget. The shown 1 GiB reserve is an example, not a measured safety guarantee.

Weights and any live reference model already consume GPU memory. The reserve must cover activations, full vocabulary logits, PyTorch gather temporaries, allocator overhead and other users. Prefill token budget and cached context length affect peak workspace. Driver-free memory also excludes memory already reserved by PyTorch's allocator, making this a conservative snapshot rather than an exact reusable-memory inventory.

The snapshot can become stale immediately. Correct block arithmetic does not prove that a future forward cannot run out of memory. Do not silently catch OOM and claim successful inference. Measure headroom on the actual Colab workload and report the chosen settings.

Tests and notebook evidence

The Colab tests cover byte arithmetic and allocation, dtype scaling, chunked prompts, mixed arrivals, EOS, zero budgets, strict admission, cancelled/preempted requests, lifetime IDs, seeded replay, pre-commit failure retry and post-commit sampling failure. Real small Llama models check HF/C2 greedy parity on CPU and CUDA. FP16 logits are compared numerically with an FP32 model loaded from the same rounded weights, using `rtol=atol=1e-2` as in C3; FP16 is not required to be bit-equal to FP32.

The full-TinyLlama notebook run reported 7,328 mismatches among 64,000 logits under that threshold, with maximum absolute difference about 0.12117. The follow-up diagnostic on that first two-row prefill showed zero paged/no-cache FP16 differences, identical cross-precision errors in both FP16 paths, and matching argmax choices in all three paths. Thus the observed first-batch error is not introduced by paging. That diagnostic alone did not validate subsequent decode steps or establish a full-model error ceiling; the complete run is recorded below.

The notebook now uses compare_precision_paths to separate the contracts. Paged/no-cache FP16 scores retain the `1e-2` relative/absolute check and must have equal argmax on every scheduled row. Each sampled greedy choice must also match the rounded-weight FP32 reference on this workload. All logits must be finite.

FP16/FP32 maximum/mean score differences and historical threshold mismatch counts remain recorded for every row, but the uncalibrated full-model cross-precision score gate is removed, not increased to an arbitrary number. This intentionally changes the validation contract: engine correctness requires paging agreement and tested greedy parity, not a claim of full FP16/FP32 logit equivalence. The original small-model numerical regression test remains. Additional tests accept baseline precision drift but reject paging errors, changed sampled choices and nonfinite scores.

The notebook uses real TinyLlama FP32 references and records every planned row, output routing, preemption, retry and final block release. FP32 logits use `rtol=1e-5`, `atol=1e-4`: the absolute allowance matches the previously observed small-model-backend differences, and argmax parity is checked separately. It records numerical ranges, not a single selected difference.

Recorded Colab validation

The complete user-supplied run on November 10, 2026 at source `d013743` passed 422 tests; one test requiring two CUDA devices was skipped. The original T4 environment, timestamp and test output are archived in D5_engine.json.

Continuous arrival finished in seven steps with 18 successful model rows. The three-block preemption case finished in eight steps with ten original rows plus two replay rows. The failure/retry case finished in seven successful steps with ten original rows plus three replay rows; failed partial execution is not included in that successful-row count. Its failure snapshot retained identical histories, progress, resident lengths and ownership before retry.

Across all 43 successful FP32 rows, no-cache/HF argmax checks passed. Per-row maximum absolute HF differences ranged from zero to 3.6239624e-5. Seeded output and RNG replay, EOS, cancellation, admission errors and exact KV pool arithmetic passed. All demonstrated pools were fully released.

The FP16 case processed five rows and generated four tokens. Paged/no-cache FP16 per-row maximum absolute differences ranged from zero to 0.015625, passing the combined `rtol=atol=1e-2` rule. All four sampled choices matched rounded-weight FP32. FP16/FP32 per-row maxima ranged from 0.00964546 to 0.12116957, retained as measurements rather than a score bound. This validates the documented workload and contract, not universal FP16/FP32 logit equivalence or OOM safety.

The archived JSON was checked against the supplied text and its internal invariants without running models locally. Existing notebook outputs were not modified. Local syntax/JSON/whitespace checks remain static only.

Three interview questions

1. Why does generating `G` tokens require only `G-1` decode forwards? The last prompt row predicts the first output. Each later forward processes the preceding output. The final prediction is returned without being processed.
2. Why must recomputation preserve token history and RNG state? It rebuilds K/V for already chosen tokens. Resampling would change the sequence and consume random draws twice.
3. Why is a correctly sized KV pool not a total-memory guarantee? The pool excludes weights and variable workspace. Free-memory readings can become stale, and temporary allocations can still exceed headroom.

Next: E1, streaming OpenAI-compatible serving driven from Colab. Prefix sharing, copy-on-write, Triton and CUDA graphs remain deferred.

21. Streaming chat over loopback HTTP (E1)

Code: app.py, local.py. Tests: test_e1_server.py. Notebook: E1_streaming_server.ipynb.

Status and scope

E1's complete user-supplied T4 run at source `cee6f49` passed on November 11, 2026: all 114 D5/E1 tests passed, including both socket disconnect-order cases. The original JSON is archived in E1_streaming_server.json. D5's complete Colab run at source `d013743` is validated under its corrected precision contract: same-precision paging checks and sampled FP32 greedy parity, with cross-precision score differences recorded rather than claimed equivalent. The E1 demo uses FP32 and a small explicit KV pool. It needs separate transport/lifecycle evidence.

The user approved loopback-only exposure: `127.0.0.1` inside Colab. There is no public tunnel, authentication, TLS, CORS or deployment. Do not bind this teaching app to `0.0.0.0` or expose it to other machines. Other processes in the same Colab runtime can access it.

Where HTTP fits

D5 accepts token IDs and returns cumulative token histories. A chat client sends JSON messages and expects text. E1 connects those interfaces:

```
HTTP JSON messages
  -> validate the supported options
  -> tokenizer.apply_chat_template(..., add_generation_prompt=True)
  -> prompt token IDs
  -> one shared D5 engine and D4 scheduler
  -> generated token IDs
  -> decoded text
  -> HTTP JSON or streamed SSE records
```

The chat template inserts the model's role markers and assistant prefix. It is not the same input as simply concatenating message text. The notebook's direct reference uses the exact same template and token IDs.

We reuse the engine. We do not load a model or allocate a KV pool for each HTTP request. A separate engine per request would lose shared scheduling.

A deliberately limited OpenAI-compatible subset

Endpoints:

Method and path	Behavior
`GET /health`	200 while healthy; 503 after an inference-driver failure
`GET /v1/models`	Lists the single configured model ID
`POST /v1/chat/completions`	JSON completion or streamed chat chunks

Supported request fields:

- `model`: must exactly match the served model ID.
- `messages`: nonempty list of `{role, content}` objects. Roles are `system`, `user` and `assistant`; content is a nonempty string. The final role is `user`.
- `max_tokens`: integer 1 through 128, default 16. The model context and single-request pool capacity can impose tighter limits.
- `temperature`, `top_p`: C1 options, default greedy temperature zero and no nucleus filtering.
- `stream`: boolean, default false.
- `seed`: optional nonnegative integer below `2**63`, creating a per-request generator on the model device.

- `n`: only the integer 1.

Unknown options are rejected, not ignored. There is no tool calling, multimodal content, custom stop sequence, logprobs, streaming usage option, Responses API or legacy Completions endpoint. This is not a claim of full OpenAI API compatibility or SDK certification.

Bodies are limited to 64 KiB. Invalid JSON, unsupported options and invalid prompt/output budgets return an OpenAI-shaped error object with HTTP 400. Oversized bodies return 413. The finite lesson lifetime quota returns 429. Inference failure returns 503 when headers have not already been sent.

What streaming means

Without streaming, the server waits for completion and returns one JSON object with `choices[0].message`, `finish_reason` and token usage.

With streaming, the response has content type `text/event-stream`. **Server-sent events (SSE)** are UTF-8 text records separated by a blank line:

```
data: {"id":"chatcmpl-tinyserve-0","object":"chat.completion.chunk",...}

data: {"id":"chatcmpl-tinyserve-0","object":"chat.completion.chunk",...}

data: [DONE]
```

The abbreviated JSON above is a diagram, not a literal request to send. Actual records contain:

1. An initial delta declaring `role: "assistant"`.
2. Zero or more deltas containing newly available text.
3. An empty delta with final `finish_reason`.
4. The literal `[DONE]` marker, only after successful completion.

The ID, creation timestamp and model remain stable within one response. The choice index is always zero. Engine `"eos"` maps to API `"stop"`; an exhausted output budget maps to `"length"`.

An SSE record is not a network packet or necessarily one model token. TCP may split or combine records. The client must parse complete lines/events, not assume every received byte buffer is one JSON object.

Why not decode each token independently?

A token may represent part of a word or part of a Unicode character. Decoding token fragments independently can insert incorrect spaces or replacement characters.

The server uses the existing Transformers `TextStreamer` buffering logic, overriding its output callback rather than printing to the terminal. It accumulates token IDs, emits text at supported stable boundaries and flushes remaining text at completion. Special tokens are skipped and tokenization-space cleanup is disabled in both streaming and nonstreaming decoding. This can delay the first text chunk until a word boundary. It is not a promise of one visible text chunk per generated token.

One owner for the engine

`ChatDriver` runs one asynchronous task. Within that task an engine forward is synchronous. No second thread or task calls the engine during that forward. Between successful steps, a small asynchronous yield allows HTTP handlers to admit new requests or cancel disconnected ones.

This preserves D4's rule: arrivals and cancellations happen only between completed forwards. It also permits different clients to share a packed model forward and the same paged pool.

This teaching choice blocks the event loop during each forward. It is not a production latency design; it does not make CUDA work nonblocking or enable cancellation halfway through a kernel.

Slow clients and retained histories

Each active HTTP job has a queue with capacity one. D5 outputs are cumulative snapshots, so a newer snapshot can replace an unread older one without losing generated token IDs. The consumer tracks how many IDs it has decoded and decodes only the new suffix.

This prevents unbounded per-token queues. It can coalesce several tokens into one visible chunk, which is correct SSE behavior.

D5 retains completed histories for inspection. Rather than silently retaining unlimited GPU histories in a long-lived server, E1 accepts at most 32 requests over one app lifetime by default. After that, it returns an explicit 429. This is a finite lesson server, not a throughput-oriented service. To start another session, create a fresh engine and app. The quota counts successful admissions, including requests later cancelled, not malformed requests.

Disconnects, failures and shutdown

When a client disconnects, the response generator's `finally` block removes its job and cancels any unfinished engine request. Resident blocks become available to other requests. Already completed requests need no cancellation. Disconnect checks and cancellation cannot interrupt a running forward. The initial role event is not an inference-progress barrier. Generation can finish before the client reads that event or its disconnect reaches the server. In that case, `"length"` remains correct; disconnecting must not rewrite a completed request as `"cancelled"`.

If engine execution raises, the driver logs the error, notifies all waiting clients, closes the engine and rejects future requests. It does not silently retry indefinitely or emit a fabricated completion.

After SSE headers are sent, the HTTP status cannot change to 503. Instead, the server sends an SSE `error` object and closes the stream **without** `[DONE]` or a successful finish chunk. Clients must treat that as failure.

Engine `close()` is a permanent abort, distinct from cancelling one request. It releases all leases even when a failed D5 plan is pending, drops retained pending logits and prevents further admission or stepping. It does not acknowledge failed computation. The fixed pool's tensors remain allocated while the engine is referenced.

The app's lifespan starts and stops the driver. The notebook's `serve_local(app)` context binds an OS-assigned `127.0.0.1` port, waits for Uvicorn readiness, then shuts down its task and socket in `finally`. There is no detached server to remember to kill, second model-loading process, fixed-port conflict or notebook event-loop patch.

Tests and evidence

Tests cover JSON/SSE schemas, stable IDs, Unicode text reconstruction, usage, EOS mapping, strict validation, byte limits, quota, shared forwards, bounded queues, cancellation and fail-closed behavior.

ASGI in-process tests validate protocol behavior, but HTTPX's ASGI transport can buffer streams. They do not prove real network incremental delivery. A separate loopback-socket test gates the driver's wake-up with an async event, without changing the production driver. It proves that the initial role event arrives while inference is paused, then disconnects and checks cancellation. A second case lets inference finish before disconnecting and requires `"length"` to remain unchanged. Both cases check pool release and shutdown. The separate driver cancellation test covers requests after inference starts.

The notebook uses a fresh T4 runtime and real TinyLlama FP32. It verifies health and model discovery over actual HTTP, streamed/nonstreamed text against direct greedy generation, concurrent clients, invalid requests and early disconnect. It records model-forward ownership and final pool state. Unlike the gated test, the real-model demo permits inference to run freely. It records cancellation or normal length completion and checks the corresponding token count; neither outcome can leak jobs or blocks. The server must stop before saving the complete result payload. For finite, predictable overlapping

workloads, the demo temporarily disables EOS stopping and restores the original model setting on exit. Deterministic tests separately verify the app's EOS-to-`stop` mapping and early release.

The archived FP32 demo matched direct token histories and JSON/SSE text. Both concurrent clients shared eight forwards. Each completed request generated eight tokens; the streamed text used three content chunks of 2-11 characters. The disconnected request was cancelled after three tokens. All 16 blocks were released, block tables were empty, and shutdown closed the engine. The fixed KV pool held 11,534,336 bytes (11 MiB).

Local syntax, notebook JSON and whitespace checks remain static only; the runtime evidence came from the user's Colab run. This lesson makes no throughput, TTFT or latency claim; E2 owns those measurements.

Three interview questions

1. Why is streaming not just returning one token per HTTP response? One response stays open. It carries a sequence of framed events; decoding and network buffering can group multiple tokens together.
2. How can disconnected clients release GPU cache safely? Cancel at an engine step boundary and release that request's leases. Do not mutate cache ownership while its forward is running.
3. How do you report failure after HTTP 200 has already been sent? Send an explicit stream error and terminate without the success marker. You cannot retroactively change the response status.

Next: E2, direct throughput, time to first token and latency versus HF, without conflating token readiness with HTTP text delivery.

22. Measure the engine against Hugging Face (E2)

Code: engine_vs_hf.py. Tests: test_e2_bench.py. Notebook: E2_engine_vs_hf.ipynb.

Status and scope

E2's complete user-supplied T4 run at source `92de867` passed on November 11, 2026: all 119 D5/E2 tests passed, with no skips. Complete evidence is archived in E2_engine_vs_hf.json. E1's separate run is archived in E1_streaming_server.json. It does not establish E2 performance.

The user selected a **direct comparison**, not HTTP timing. TinyServe uses the public D5 engine; HF uses cached greedy `generate()` with DynamicCache. Both run on the same device, with the same weight precision, prompt IDs and output count. The notebook copies the HF weights strictly into TinyServe. The installed HF attention implementation is recorded, not assumed equal to TinyServe's PyTorch paged-gather implementation.

This is a finite simultaneous-arrival cohort, not a loaded production server. There is no network, SSE, tokenizer, text decoder, arrival-rate sweep, public binding or deployment change.

Define the boundaries before measuring

An **output token** is a selected token ID, including any EOS ID. EOS stopping is disabled for this benchmark so every request produces exactly `G` outputs. It is a fixed-work experiment, not ordinary conversational stopping behavior.

All `B` requests share a start time. TinyServe starts timing before admission; HF starts before `generate()`. The prompt batch is already on the GPU. CUDA is synchronized before starting and whenever output IDs are observed. These are instrumented wall-clock measurements, including Python overhead and synchronization, not isolated GPU kernel times.

Metric	Definition
TTFT	Time from common start until the first output token ID is available
Request latency	Time from common start until that request's last output token ID is available
Inter-token latency (ITL)	Difference between consecutive output-ID timestamps for one request
Total time	Time until the full generation call/engine drain returns, followed by CUDA synchronization
Output throughput	`B * G / total_seconds`; prompt tokens are not counted

Request latency ends at **last ID readiness**, not receipt of an HTTP finish record. Total time additionally includes final return/drain work. All requests arrive together; a later TinyServe request can wait behind earlier chunked prefill, and that wait belongs in its TTFT.

Example: two requests produce three tokens each. Their relative timestamps are `[0.1, 0.3, 0.5]` and `[0.2, 0.4, 0.7]` seconds; the full run takes 0.8 s.

- TTFT: 100 ms and 200 ms.
- Request latency: 500 ms and 700 ms.
- ITL: `[200, 200]` ms and `[200, 300]` ms.
- Throughput: six outputs / 0.8 s = 7.5 output tokens/s.

For `G=1`, TTFT equals request latency and there is no ITL. We store `null` for an absent ITL summary rather than dividing by zero or inventing a value. Invalid, nonfinite or backwards timing raises an error; it is never clamped.

Observe one full run

A3 estimated decode latency by subtracting a separate one-output run from a full run. Those runs can have different noise. E2 instead records every output timestamp **within the same full run**.

TinyServe records the updates returned by `engine.step()`. Partial prefill does not create an output timestamp. Requests updated in the same step get the same observation time. HF receives a lightweight streamer: its first `put()` is the input prompt batch and is skipped; each later `put()` contains one selected token per batch row. No text decoding runs in that callback. The returned HF sequence is checked against the stream's recorded IDs.

This observer costs time. Synchronization and Python callbacks can affect the two implementations differently. Do not compare these numbers directly to an uninstrumented `generate()` run, claim HTTP TTFT, or interpret a callback timestamp as the exact instant a CUDA kernel selected a token.

Keep work and memory explicit

Prompts have equal lengths and no padding. Notebook batch rows are identical, with no shared prefix cache. Every request still owns independent KV blocks. Tests also use different rows to verify routing.

With prompt length `P` and output budget `G`, each request processes `P + G - 1` positions. For block size `S`, the pool has `B * ceil((P + G - 1) / S)` blocks. This rounds **per request**, not after adding all requests together. Every request fits simultaneously; any preemption or leaked block is an error in this no-pressure benchmark.

The notebook measures `P` in `{32, 128}`, `B` in `{1, 4}`, `G=16`, with 16-slot blocks and a 128-row forward budget. The largest pool is 36 FP16 blocks: `36 * 360448 = 12976128` bytes (12.375 MiB). This is only KV storage; weights, activations, logits, gather temporaries and allocator memory remain.

TinyServe allocates a fresh fixed pool **before** timing each run, as an already-started serving engine would. HF allocates its dynamic cache inside `generate()`. This lifecycle asymmetry is explicitly recorded. It is not a claim of equal cache-allocation work or equal attention kernels.

Both models remain resident while measuring. CUDA records baseline and peak allocated bytes plus their difference. TinyServe's baseline includes its preallocated pool; HF's does not. These are allocator allocation observations, not driver-free memory, reserved bytes or standalone model memory. Freed allocator blocks can be reused; the benchmark does not flush them per trial.

Trials, correctness and conclusions

One full-length warmup runs per mode, outside reported measurements. Three measured trials alternate TinyServe-first and HF-first order. The result retains every trial and reports minimum, median, maximum and sample count. TTFT and request-latency summaries pool requests across trials; ITL summaries pool intervals. Those samples are correlated, not independent confidence intervals. Three trials do not justify a production p99.

Each trial records `HF total time / TinyServe total time`. Values below one mean TinyServe is slower. There is **no speedup gate**. This readable Python engine is not expected to beat optimized HF kernels at every batch size.

The notebook first runs a short full-model FP32 greedy-parity gate, then converts both models to FP16 for timing. Fixed token counts are always required. FP16 token equality is recorded per request/trial, not assumed. If trajectories diverge, compare fixed workload cost, not semantically identical continuations or universal numerical equivalence. D5's precision contract still applies; no new FP16 score tolerance is invented.

EOS configuration is restored even on error. HF gets a fresh generation configuration with no EOS stopping or extra sampling processors. Modern HF merges unset fields from `model.generation_config`, so the benchmark temporarily uses the fresh policy as that default source too. It restores the original object in `finally`, including after generation or timing failure; the original policy's contents are not edited. No `use_model_defaults` argument is passed. Tests install conflicting sampling/beam/penalty settings in the original policy and check isolation, fixed work and restoration.

Engine cleanup closes and releases even a failed pending forward. Errors propagate; no failed trial becomes a fabricated timing result.

Recorded Colab result

The FP32 gate and every measured FP16 continuation matched HF. All timed engine runs recorded zero preemptions and full block release. HF used SDPA. All summary statistics were recomputed from the supplied raw trials.

Output throughput is shown as **minimum / median / maximum**:

B	P	TinyServe output tokens/s	HF output tokens/s
1	32	12.13 / 14.36 / 14.54	33.12 / 38.23 / 40.62
4	32	22.20 / 24.68 / 25.13	101.97 / 118.16 / 121.91
1	128	13.69 / 14.38 / 14.57	38.27 / 40.60 / 41.64
4	128	19.74 / 21.29 / 22.77	101.09 / 130.26 / 137.19

HF was faster in all four cases. Median TTFT was 71.79/29.07, 162.22/36.62, 89.22/29.18 and 394.10/72.22 ms (TinyServe/HF), respectively. For the largest case, TinyServe TTFT spanned 68.32-663.79 ms across requests and trials, consistent with staggered prefill readiness. Profiling would be required to attribute the performance gap to individual operations.

The complete archive preserves original timestamp, source and environment. Local checks remained static/data-only; no benchmark or model ran locally. These results do not establish HTTP latency, production p99 or universal FP16 token equality.

Three interview questions

1. Can higher throughput mean worse TTFT? Yes. Batching can increase total outputs per second while later requests wait for shared prefill work.
2. Why measure output times within one run? It avoids subtracting unrelated noisy runs and preserves individual request delays and decode intervals.
3. What makes a fair comparison more than equal model names? Match weights, precision, prompts, output budgets and timing boundaries. Record backend, cache lifecycle, observer costs and token divergence.

Next: H packaging. After E2 and packaging, pause to review the entire core project properly before any Parts F/G stretch work.

Stop here: review the core project

E2 is validated. After the H documentation and PDF checks, **do not start Parts F/G yet**. The user requested time to go through the whole core project properly first. Use this checklist as a discussion guide, not a request to execute models locally.

A. Start from tokens and hardware

- Explain token IDs, embeddings and why the final logits row predicts the next token.
- Trace one prompt through `(N,) -> (N, 2048) -> (N, 32000)`.
- Explain decode weight traffic, prefill reuse and why batching can improve throughput.
- Distinguish datasheet peaks from measured sustained bandwidth/compute.

B. Rebuild the model mentally

- Explain RMSNorm, RoPE, causal attention, GQA and SwiGLU in order.
- Draw query/key/value shapes: 32 query heads, four KV heads, head dimension 64.
- Trace both residual additions in one decoder layer and the final model norm.
- Explain strict HF weight loading and what FP32 parity does and does not prove.

C. Generate without changing semantics

- Explain greedy selection versus sampling and a random draw.
- Trace EOS, zero output budgets, seeded replay and strict context limits.
- Explain why causal K/V can be reused and why a cache does not store final text.
- Derive the difference between `G` returned outputs and `G-1` decode appends.

D. Own and schedule memory correctly

- Map logical positions to physical blocks and slots, including scrambled tables.
- Calculate FP16 KV bytes/token: `2 * 22 * 4 * 64 * 2 = 22528`.
- Distinguish whole-block budget rounding, unused last-block slots and free blocks.
- Explain reserve, forward commit and scheduler acknowledgement separately.
- Walk through a new arrival, chunked prefill, mixed decode and newest-victim preemption.
- Explain why replay rebuilds K/V without sampling or consuming random draws.
- Trace forward failure/retry versus failure after committed logits.
- Explain why cancelling returns leases without shrinking fixed GPU tensors.

E. Serve and measure honestly

- Trace messages through the chat template, admission, engine, token IDs and text.
- Explain SSE framing, Unicode/word buffering, errors after HTTP headers and `[DONE]`.
- Explain the disconnect/completion race and why `"length"` can remain correct.
- Define TTFT, ITL, last-ID latency, total duration and output throughput.
- Explain the 128-row forward budget versus total resident KV capacity.
- Interpret the four E2 cases, HF's advantage and the cache-lifecycle asymmetry.
- Distinguish observed token equality from universal numerical equivalence.

Evidence and limits

For each claim, open the relevant guide chapter, source, test and complete recorded result. Explain what the test actually exercises. Suite counts overlap, and a passing local static check is not a new GPU runtime validation.

The loopback app is a finite teaching server, not a public production service. Prefix caching, Triton attention, CUDA graphs, speculative decoding, another model family and INT8 KV are not implemented core features.

Proceed to stretch work only after the user finishes this review and explicitly chooses the next lesson. Do not mark the review complete merely because H built.